Predictive Spice-Stock Preservation System for Warehouse Environments

Mohammad Faraj

23077476

<https://github.com/MoFa-14/warehouse-spice-risk.git>

UFCFXK-30-3

Digital Systems Project

UWE Bristol University of the West of England

Abstract

Storage conditions in warehouses can differ from one area to another in the same facility, which can provide difficulties with regard to moisture-sensitive items like spices. This project seeks to solve this issue through the creation of a prototype of a predictive warehouse monitoring system to support immediate environmental risk awareness and operational decision-making.

The finished prototype was designed using an IoT architecture composed of four layers. The sensing layer includes a semi-squared sensor pod that is made using an Adafruit Feather nRF52840 Express board and a Sensirion SHT45 temperature and RH sensor. The communication layer not only provides BLE communication with the physical sensor pod, but also includes a synthetic sensor pod to facilitate the testing of multiple sensor pods. The storage layer logs telemetry and forecast data to a local SQLite database. The application and control layer provides a Flask-based dashboard for visualisation, rules-based alerts and short-term forecasting (next 30 minutes) of environmental conditions.

The intended deployment model involves multiple physical sensor pods distributed across separate zones within a warehouse. For the evaluation of the prototype, however, only one primary sensor pod and one synthetic sensor pod were tested, thus examining multi-source ingestion, communication, forecast support and end-to-end software operation under varying conditions. Therefore, this system should be viewed as an integrated, experimental sensor pod system for the monitoring of warehouse environmental conditions and short-term forecast risk, before full industrial deployment within a warehouse environment.

Acknowledgements

I wish to thank my supervisor, Dr Neil Phillips, for supporting me throughout the course of this project, and for his crucial advice on designing the hardware pod and facilitating my connection to electrical technicians at the School of Engineering to have the board and sensor soldered together.

Table of Contents

Abstract 1
Acknowledgements 2
Table of Contents 3
Table of Figures 4
Introduction 5
Literature Review 8
Design 11
Implementation and Testing 19
Project Evaluation 23
Further Work and Conclusion 27
Glossary 28
Table of Abbreviations 30
References / Bibliography 31
Appendix A: First Appendix 35

Table of Figures

Figure 1. Warehouse micro-zones in the storage environment 5
Figure 2. Four-layered IoT system architecture 6
Figure 3. Hardware pod including the Feather board, the SHT45 sensor, and the battery 11
Figure 4. Wiring and connections between the board and the sensor 12
Figure 5. PRT Indoor Air Temperature Sensor 12
Figure 6. Telemetry and gateway communication sequence 13
Figure 7. SQLite schema and data model 14
Figure 8. Dashboard view architecture 15
Figure 9. Forecast input/output logic. 16
Figure 10. Forecast-driven alert and recommendation flow 17
Figure 11. Back test forecast accuracy using MAE and RMSE across pods and forecast scenarios 25
Figure 12. Dashboard main overview page 35
Figure 13. Health page 36
Figure 14. Alerts page 36
Figure 15. Readings page 37
Figure 16. Review page 38
Figure 17. Gateway running in the terminal 39
Figure 18. Forecast running in the terminal. 39
Figure 19. Synthetic pod running in terminal 40

Introduction

Environmental Nature of Warehouses

Due to the multitude of airflow, door openings, stock movement, ventilation and external weather conditions that exist within warehouses, many micro-zones exist where environmental conditions (such as temperature and RH) differ significantly between environmental conditions found in separate parts of the same warehouse (Zhang, 2025). For example, in the case of atmospherically sensitive products (e.g. spices), these environmental micro-zones can lead to increased uptake of moisture, caking and/or loss of aroma (Navarro, 2018; Voelker et al., 2020). Figure 1 illustrates this issue by illustrating how different warehouse zones experience different conditions simultaneously.

Figure 1. *Warehouse micro-zones in the storage environment*

This project develops a prototype of a predictive warehouse monitoring system that will collect data regarding the environmental conditions in the various storage areas and transmit that telemetry to a central point (the gateway) where the collected data will be stored and used to create forecasts for short-term decision-making and to allow for proactive intervention. This monitoring, logging of data, forecasting and actionable feedback mechanism will enable warehouse staff to proactively respond to adverse conditions in such a manner that will prevent them from continuing long enough for the stored spices to be damaged by the adverse conditions (Parkhi et al., 2022; Lao et al., 2011).

Objectives

The goals of this project are to develop an IoT-based predictive system that will allow for environmental conditions to be monitored and predicted so that warehouse-stored spice inventory can be preserved. The specific goals of this project are:

1. Building a physical sensor pod that measures temperature and relative humidity in a warehouse.

2. Developing a communication method for the sensor pod to send telemetry to a gateway reliably using BLE.

3. Building a second, synthetic sensor pod that is used to develop data consistent with actual warehouse conditions and test the functionality of multiple sensor pods connected to the same gateway.

4. Creating a storage repository that stores the raw telemetry data, system state and forecast output data in a structured and queryable manner.

5. Creating a forecasting subsystem that uses recent sensor telemetry to forecast short-term future temperature and relative humidity conditions.

6. Generating alerts and/or operational recommendations for both current and forecast conditions.

7. Creating a dashboard that presents sensor readings, trends, alerts and predictions in a clear and easy-to-understand manner.

8. Evaluating the sensor pod and all other subsystems as an integrated system, including sensing, communication, storage, forecasting and dashboard operation under realistic project constraints.

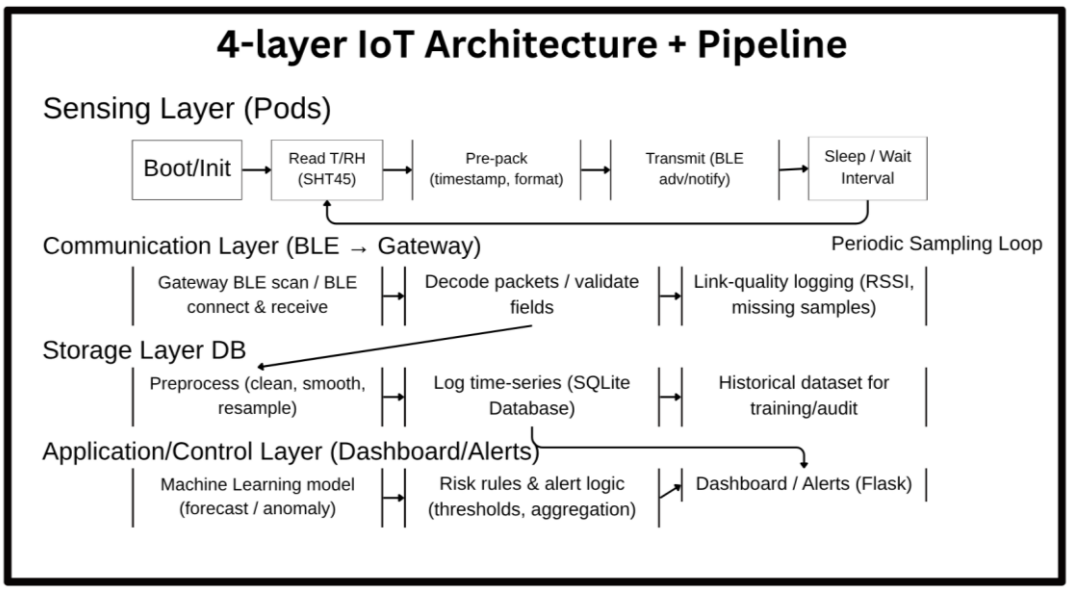


***Figure 2**. Four-layered IoT system architecture*

Figure 2 summarises the four-layer architecture that structures the remainder of the report. The following sections explain the literature basis, design decisions, implementation, testing, and evaluation of the prototype, where extra details related to the working dashboard and extra test cases can be found in the appendix (da Costa, 2022).

Literature Review

Real-time monitoring is adopted in food supply chains, mainly in storage and shipping, where there is considerable opportunity to sustain the quality of food (da Costa, 2022). In a warehouse, IoT technologies that monitor air allow for the management of food inventory due to early detection of adverse temperature/humidity conditions, as well as providing timely corrective action to decrease waste (Parkhi, 2022). The IoT architecture should be a four-layered system that provides a framework to analyse designs and compare performance end-to-end (da Costa, 2022).

End-to-end architecture, edge gateway, and logging

The implemented end-to-end IoT architecture consists of sensing, communication, storage, forecasting, and dashboard-based alerts/recommendations. This approach is aligned with streaming networks and allows for a clear overview of an organisation's operational visibility and its model governance (Khattach, 2025; Carnier, 2025). The gateway functions as an Edge Computing (edge-Gateway), enabling collection, filtering and analysis of data collected near the sensors through low-latency connections, before sending that data to external services (Zhang, 2025; Gowrishankar, 2023). Decision-support for operations is improved when warehouse technologies use the same model for data capture and transmission as described in the concept of a Pod, Gateway and Dashboard (Lao, 2011).

Why temperature and RH are sufficient for spices, and why prediction helps

An emphasis on temperature and relative humidity (RH) is justified since they are the primary variables to consider when monitoring the condition of foodstuffs, especially those sensitive to moisture (da Costa, 2022). Temperature and RH affect the degree of water activity and potentially degrade quality (Navarro, 2018). The materials-science literature demonstrates that both temperature and RH are the factors for moisture sorption, deliquescence threshold and glass transition behaviour, which can cause physical destabilisation, like cracking (Voelker, 2020). Other than mould damage, the loss of aroma/volatile compounds under improper storage conditions can significantly reduce the organoleptic quality of spices, even if they appear "dry" (Navarro, 2018). Framing of risks supports the idea that RH at ~65% and elevated temperature conditions are deterioration zones for spices (Navarro, 2018). Since it's recognised that hermetically sealed/vacuum-packaged/CO_2-enriched storage methods are more effective in maintaining quality than aerated alternatives, IoT technologies that provide early warnings of storage degradation allow for corrective actions without the need to change storage.

Motivation for prediction is that short excursions can cross stability boundaries (e.g., Tg/RH_0 crossover), thus rapid and potentially irreversible physical changes; therefore, using proactive monitoring with ML capabilities promotes early intervention (Voelker, 2020).

Optional psychrometric features (dew point) for stronger interpretation

Although T/RH are primarily sufficient, psychrometric indicators provide information based on these variables, enabling more measurements without extra sensors (e.g., dew point). It's estimated using

T/RH that psychrometric equations were developed through comparisons with psychrometric reference data, allowing for the reliable estimation of dew point from T/RH at the gateway preprocessing level (Górnicki, 2017).

Sensor placement and dynamic warehouse behaviour

Physical environments contain spatial variability throughout, making it difficult for a single point measurement and/or fixed thresholds to detect localised occurrences. To improve detection accuracy of micro-zones, multiple pod deployments at locations along the route taken by products to-and-from warehouse gates/vents/identified hotspots proved effective in improving detection accuracy when used in conjunction with airflow analysis and understanding of temperature and humidity sensitivity to product types (Zhang, 2025). Furthermore, dynamic environments create the potential for baseline drift caused by changes in operation (i.e., daily work), creating a need for ongoing adaptation and managing false alarms on a large scale (Zhang, 2025; Kothamali, 2025).

Hardware choice and measurement quality

The selection of hardware used affects both sensing fidelity and effective data communication through the modification of sensor connection requirements. For example, using the Feather nRF52840 with SHT45 has a higher level of measurement performance than any other component combinations used in prior studies, such as ESP8266 + DHT11, which were dependent on using Wi-Fi for connectivity and less accurate measurements than the SHT45 sensor combination (Adafruit Industries, 2019; Adafruit Industries, 2017; Espressif Systems, n.d.; Components101, 2021). Additionally, the accuracy of the data collected from the sensors will be strengthened through calibration and compensation via software (e.g., curve fitting and offset adjustments), filtering, and smoothing capabilities that stabilise the data readings, providing a stable foundation for reliable ongoing monitoring and subsequent machine learning model creation (Wang, 2020; Zhang, 2025).

Communication design: BLE pods, reliability, and scalability

For battery pods, BLE is preferred over Wi-Fi because Wi-Fi is energy-intensive and requires each node to manage IP connectivity; radio power outcomes depend strongly on duty cycle, reconnection behaviour, and how long radios remain awake (Yuksel, 2020; Irvine, 2016). This motivates controlling transmission frequency and payload size while meeting monitoring needs (Yuksel, 2020), and supports BLE pods feeding a mains-powered gateway as a scalable pattern (Yuksel, 2020), (Irvine, 2016). For large areas, BLE Mesh is a plausible extension to address coverage, with experimental guidance to keep adjacent hops short (≈15 m) to maintain RSSI and reduce packet loss (Mahmud, 2024). Reliability matters directly because packet loss → missing samples → degraded forecast reliability, so PLR/missingness should be logged and analysed as part of the data-quality pipeline (Mahmud, 2024).

Warehouse RF conditions can further degrade BLE performance: metal infrastructure and moving stock can distort propagation and reduce reception quality (Ramires, 2024). Because BLE and Wi-Fi share the 2.4 GHz band, coexistence interference can increase retries and missing samples, motivating logging and future work on coexistence-aware strategies (Eltholth, 2023). On the gateway side, scaling many BLE peripherals can introduce scheduling/resource-overlap issues that harm QoS even when devices appear connected, reinforcing the need to design conservatively for node count and traffic

load (Kim, 2025). Practical diagnostics can include BLE sniffing using an nRF52840 USB dongle to inspect traffic and debug mesh forwarding and losses (Mahmud, 2024). Separately, multi-node time-series integrity requires handling clock drift via lightweight application-layer time synchronisation (e.g., gateway beacons and timestamp correction) to prevent misalignment across pods (Li, 2015).

Forecasting evaluation, drift, and decision support

Forecasting accuracy is evaluated using MAE and RMSE through sequential backtesting over 3-hour input windows and 30-minute forecast horizons (Kempelis, 2023). Static offline models can degrade over time due to drift, so logging provides a mechanism for monitoring changing conditions, reviewing forecast performance, and supporting later model updates. The implemented system uses rule-based severity thresholds for alerting, while the forecasting component predicts future temperature and relative humidity conditions (Carnier, 2025). Simple, mobile-friendly interfaces, along with robust logging, are a priority for practical implementations of forecasting systems, even during connectivity outages (Mohd, 2022). Alert fatigue is a well-documented risk as these systems are deployed at scale; temporal aggregation of alerts is a straightforward method for mitigating this risk (Kothamali, 2025). Finally, Knowledge-based Reasoning methods, including CBR and Fuzzy Logic, can increase traditional forecasting methods by providing means to convert prior experiences into evidence-based recommendations, and the evaluation of performance can examine both Forecast Error and Operational Metrics aligned with Operational Performance (Lao, 2011).

Governance context for monitoring

For assisting with appropriate monitoring, the Governance Context of Food Safety shows microbiological quality of spices depends on FSMS Adoption/Packaging Type. Adopting FSSC 22000 v6 at the standard level aligns with other governance expectations of monitoring systems, as being able to audit. It requires a microbiologically focused and risk-based environmental monitoring programme with evaluations, trend analysis, and periodic review (FSSC, 2025). The Governance Context is a recognition of the organisation's leadership/supervision, and the functional and operational structures with a focus on management, operational, and consumer/retailer (Karam, 2021; FSSC, 2025).

Although platforms such as Adafruit IO provide quick feed-based logging and visualisation, this project instead uses a Flask dashboard with an SQLite database to provide greater control over data handling, storage structure, and integration with the forecasting and alerting pipeline (Adafruit IO, n.d.; Khattach, 2025; Peddoju, 2020; Hatem, 2022).

Design

Figure 2 illustrates how the system utilises a 4-tier architectural design of sensing, communication, storage and application/control (da Costa, 2022). The separation of concerns provided an improved means for developing, testing and extending the prototype; the sensing layer acquires measurements, the communication layer transmits and authenticates telemetry data, the storage layer retains both current and past states, whilst the application/control layer produces dashboard reports, forecasts, alerts, and recommendations (Martin, 2019).

Sensing Layer and Pod Hardware

Using an Adafruit Feather nRF52840 Express and Sensirion SHT45 in the physical pod provide a combination of accuracy measurement, BLE connectivity and low-level power operation (Adafruit Industries, 2019; Adafruit Industries, 2017). Thus, precisely measuring both temperature and relative humidity, which are then transmitted to a gateway as structured telemetry data (da Costa, 2022; Zhang, 2025; Gowrishankar, 2023). Heavy processing is explicitly not done on the physical pod, but rather on the gateway. Figures 3 and 4 depict the assembled physical pod as well as the wiring of the board to the sensors to achieve this design.

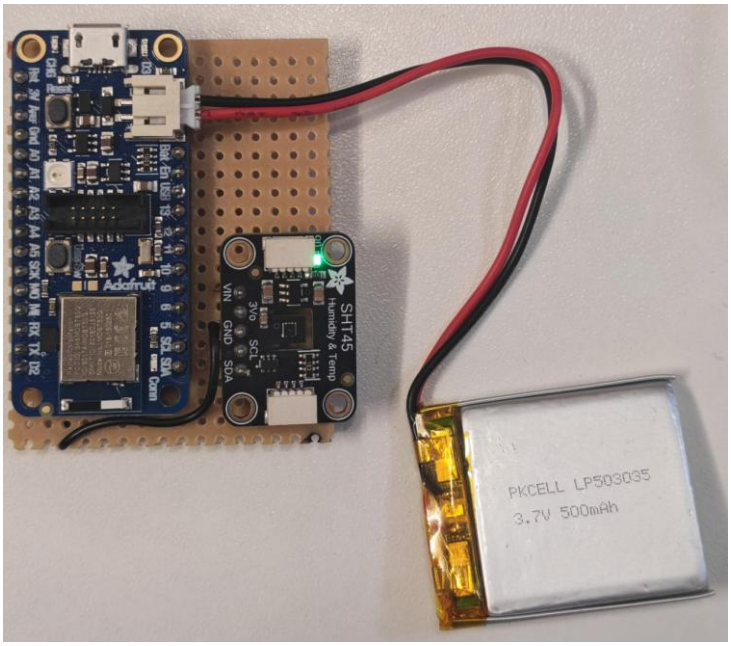


***Figure 3**. Hardware pod including the Feather board, the SHT45 sensor, and the battery*

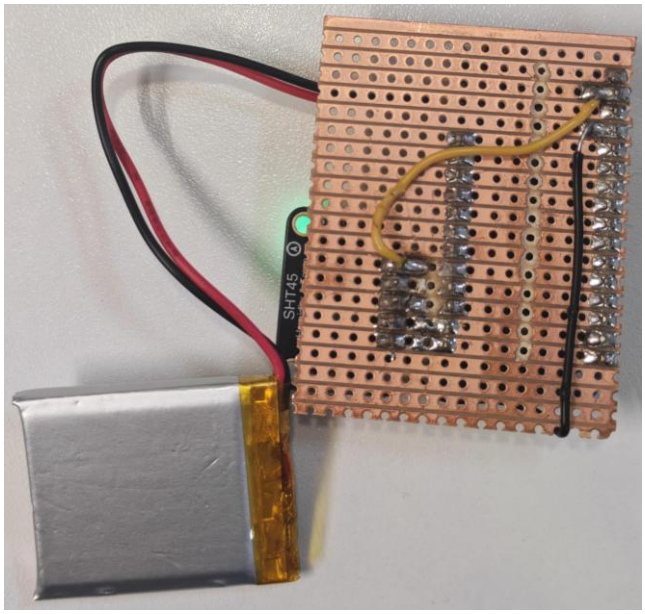


***Figure 4**. Wiring and connections between the board and the sensor*

The firmware has undergone a straightforward repeated series of stages in the following cycle: Sensor initialisation, sampling, packet preparation, transmission, and wait state. This limited complexity and increased the reliability of the system during testing, as well as assisted in identifying deployment errors. A 3D printed enclosure should provide the sensor access to ambient air while also providing protection to the board and battery, and enabling the pod to be mounted in a warehouse zone. Such an enclosure might look something similar to the "PRT Air Sensor" shown in Figure 5, but with an opening for the USB-charging socket and mounting holes.

***Figure 5.** PRT Indoor Air Temperature Sensor (Process Parameters, 2025)*

Communication Layer

The gateway receives a custom GATT Service Via BLE from the Physical Pod. Each Telemetry Packet sent from the Physical Pod contains the Temperature, Relative Humidity and Sequence information. This allows the gateway to detect when there is a missing sample or when there are duplicate samples. This communication path, from the sampling of a sensor to the validating and logging of data by the gateway, is shown in Figure 6.

Through the gateway's TCP connection, the synthetic pod sends structured data, as JSON messages, to effectively simulate a secondary warehouse area using cyclically replicated day and night behaviours, seasonal changes, and simulated communication faults (such as jitter, lost packets, and reconnections) (Aloe and Maio, 2025). Testing of multiple-pod ingestion and forecasting behaviours can occur via both pods.

BLE was favoured for the sake of the physical pod since it is battery-constrained and only needs short-range indoor connectivity to a neighbouring gateway. Doing so would avoid increasing radio and network-management overhead at each pod by utilising Wi-Fi direct, plus this prototype does not require the long-range deployment model that would have otherwise justified the use of LoRa (Wi-Fi Alliance and LoRa Alliance, 2019). Hence, the design was gateway-centred, allowing for low-power pods to maintain simplicity while the mains-powered host provided validation, storage, forecasting, and dashboard logic (Yuksel, 2020; Irvine, 2016).

The prototype currently utilises a gateway that is hosted on the developer's laptop. This laptop serves as the main Host for receiving, verifying, storing, predicting, and using the data to generate dashboards from all the telemetry. This was appropriate for developing and systematically evaluating; however, in an actual distribution centre deployment, the gateway would be better suited as a fixed device that would be permanently located on site in order to reduce the reliance on a portable development workstation and provide an increased level of continuous, reliable service.

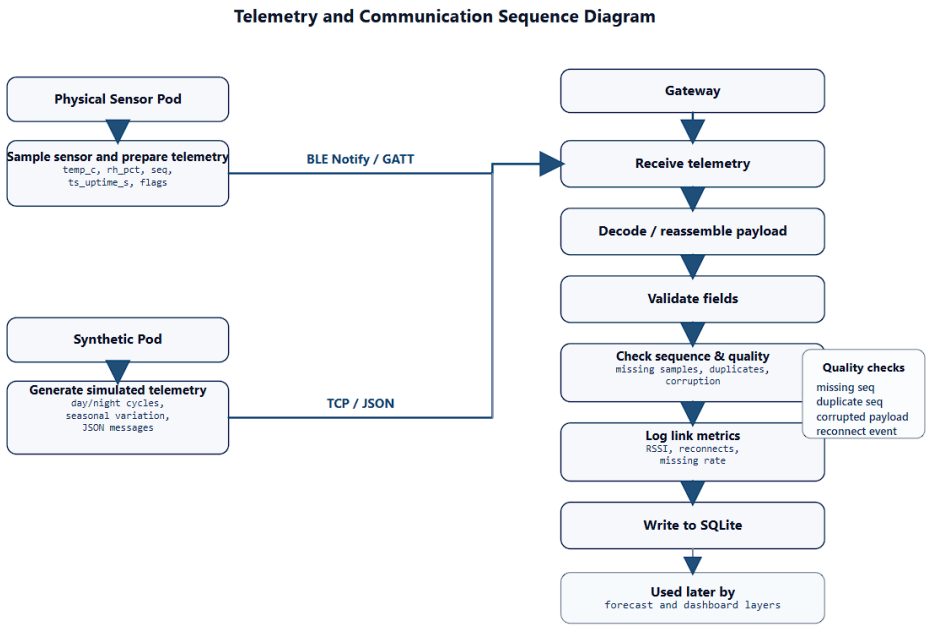


Figure 6. *Telemetry and gateway communication sequence*

Storage Layer

CSV logging was originally favoured for project development due to its easy visibility during debugging; however, SQLite was determined to be suitable as the main local data store for the finalised prototype due to its abilities as a structured, queried database without the need for an external database server. Figure 7 includes telemetry, link-state data, forecast output and assessment data stored in a single node to be used by the gateway and dashboard. This decision made sense for use in the lightweight prototype, but it is unlikely to provide the necessary evidence of suitable high-write, multi-user, and real-time production scalability ('Appropriate Uses For SQLite', n.d.; 'SQLite User Forum: Multiple Writers', 2025).

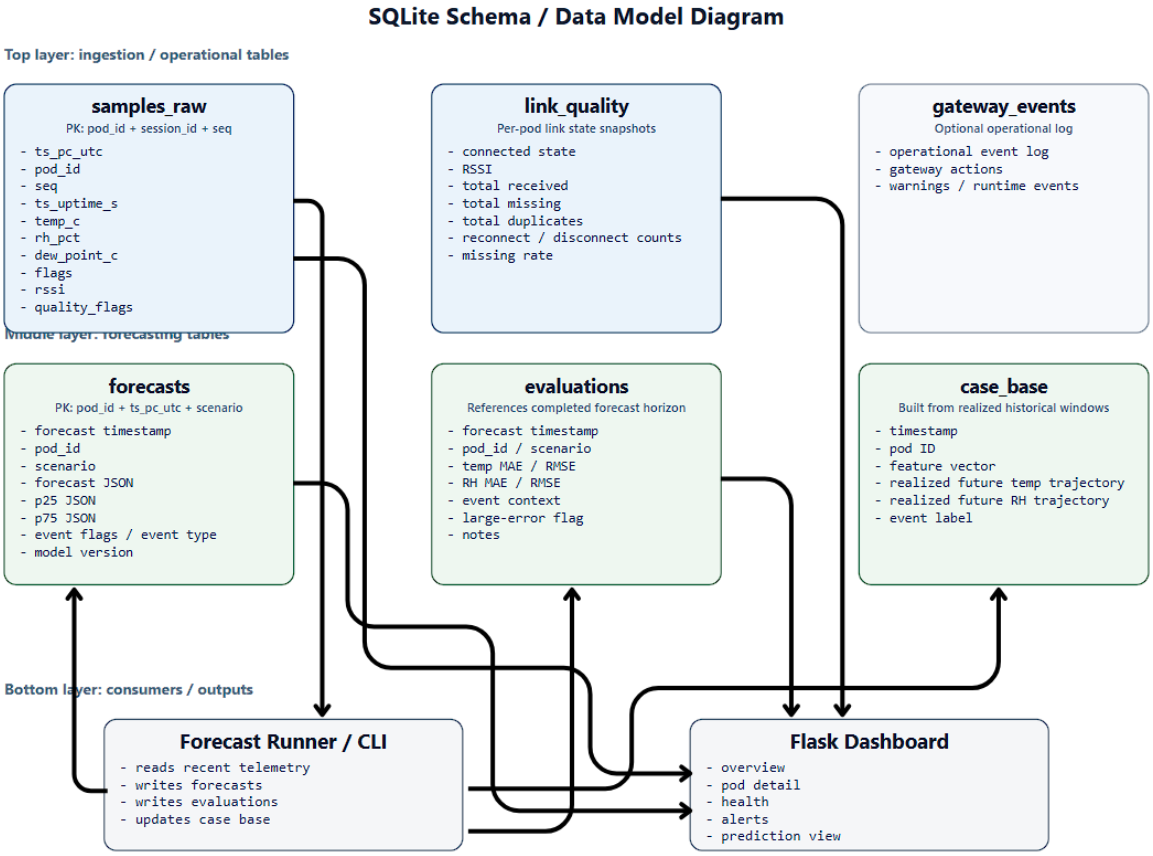


Figure 7. *SQLite schema and data model*

To execute the project, the plan included very low-inertia signal conditioning, aligning the project data with the discussions of calibration and stabilisation. In addition, there is the option of having individual software offsets applied to both the temperature and RH on downstream readouts; rolling mean smoothing is also an available option for the rendering of dashboards and the preparation of forecast input (Wang, 2020; Zhang, 2025). The dewpoint has been classified as a derived value and will be re-computed from the corrected temperature and relative humidity values if those adjustments are enabled (Górnicki, 2017). A formal lab calibration of the SHT45-based pod against a certified instrument, as a reference, was not carried out. Thus, any measurements used in this project should be regarded as uncalibrated beyond the manufacturer's specification, so extra future work is required

to perform formal metrological validation. However, the project implements software-based calibration offsets, smoothing, and dew-point recomputation in downstream processing.

Control and Application Layer

Flask was used to implement a user-oriented interface for the dashboard so that it is part of the same codebase as the other components of the prototype being designed. This allowed easy integration between both the data storage and forecasting pipeline, as well as provided flexibility over how results will be displayed. Both the architecture of the dashboard component and its relationship to the data repository are illustrated in Figure 8 (Peddoju, 2020; Hatem, 2022; Khattach, 2025).

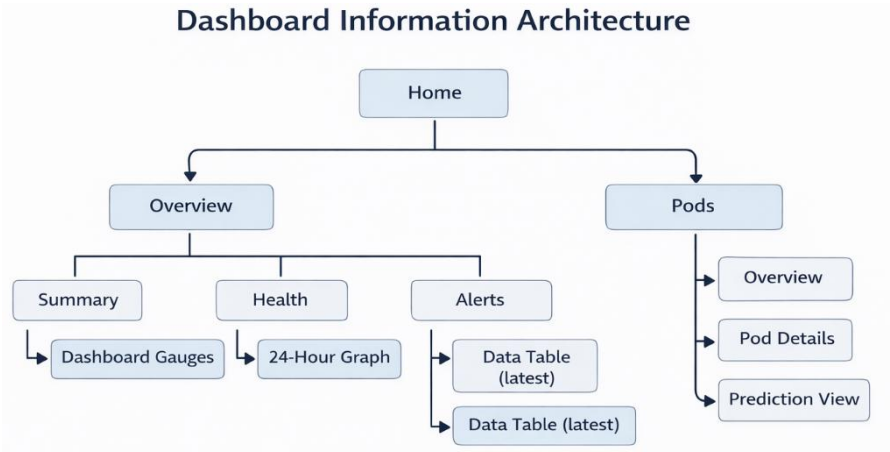


Figure 8. *Dashboard view architecture*

Forecasting Design

Instead of using attributes and characteristics to categorise behaviours (e.g., categories of rainfall—heavy, moderate, or no rain), the Forecasting Subsystem operates as a continuous value; a short-term predictor, utilising the last three hours of telemetry. This window was selected to capture recent trends and disturbances without reducing the forecast's short-term relevance. Then, transforming onto a common one-minute time grid, to project values (temperature and relative humidity) over the next thirty minutes, plus the dewpoint value derived from these forecast variables, to assist interpretation of moisture risk (Figure 9). Thus, Feature Engineering will be performed by taking the three-hour telemetry data set and down-sampling it to one-minute data sets. And calculate the dew point using the temperature and the relative humidity, and extract slope, spike and threshold context indicators using only the most recent time period (Kempelis, 2023; Górnicki, 2017; Kempelis, 2023, Nentwich and Reinhart, 2021; Cristofanelli et al., 2023; Kulanuwat et al., 2021; Jablonski and Mendrok, 2025).

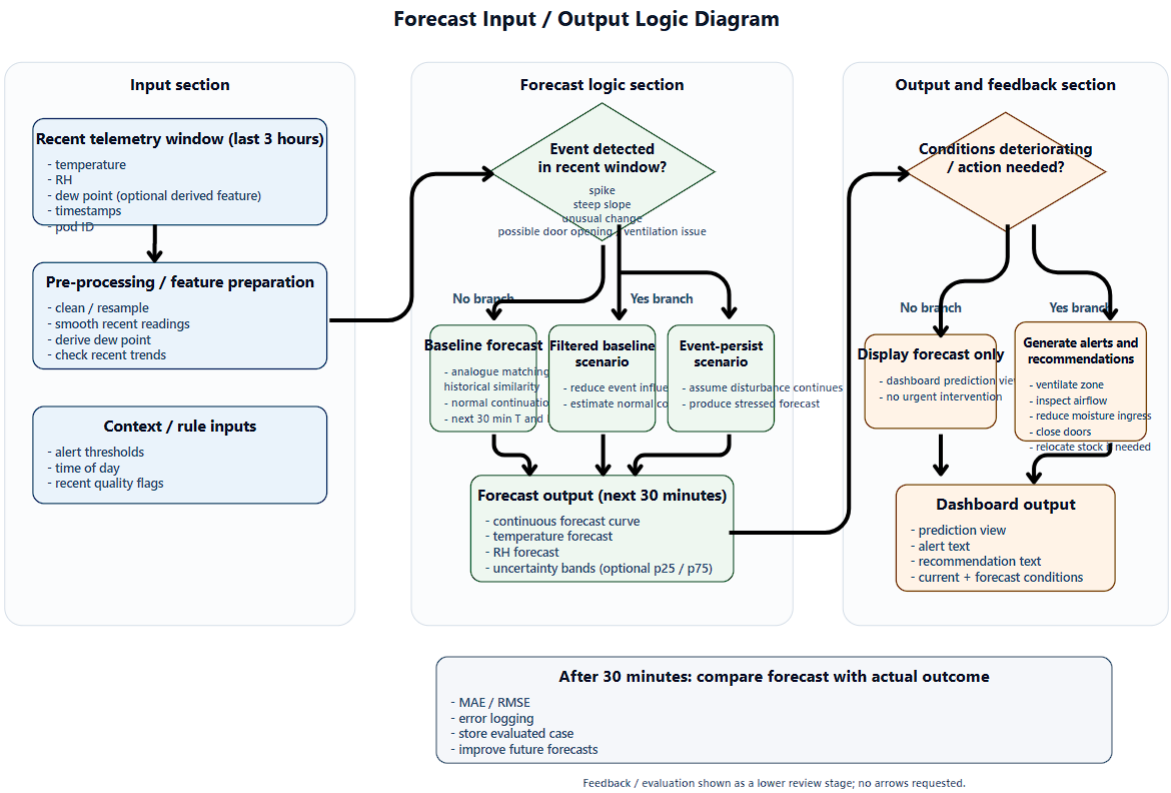


Figure 9. *Forecast input/output logic.*

Before generating forecasts, the system scans the recent time frame for disturbance use by utilising slope inspection, threshold context checking and spike detection methods to avoid misclassifying transient operational event activity as normal background behaviour (Carnier, 2025). Once the recent time frame is confirmed to contain stable behaviour, the system issues a short-term forecast indicative of baseline activity. When a disturbance has been identified, the system separates recent anomalous activity from the long-term observed trend, using historical analogue match methodology to assist with the determination of future behaviour, and where insufficient historical data exists, utilises part-performance-based persistence for future predictions, see Figure 10 (Murph et al., 2025). A lightweight forecasting process was selected over reliance on conventional threshold-based alert systems, as previous studies demonstrated conventional alert systems did not provide a reasonable near and/or accurate future activity scenario to assist operators with forthcoming operational decisions, and, likewise, current forecasting models would not be reasonably justifiable for the small student-based prototype development. Threshold-based alerting is the main baseline for this prototype, which does not employ a prediction for future values (Carnier, 2025). When a relevant historical analogue cannot support a specific analogue's prediction, a persistence forecast is the simplest baseline option available to use as a fallback assumption. To improve usefulness beyond reactive threshold warnings alone, the forecasting stage was selected by showing a thirty-minute-ahead scenario (Kempelis, 2023; Lao, 2011).

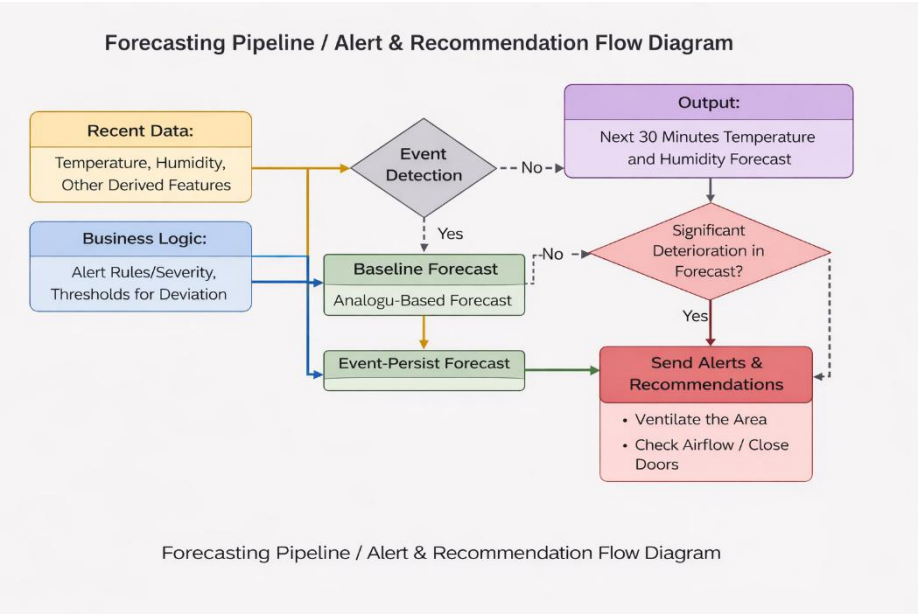


Figure 10. *Forecast-driven alert and recommendation flow*

Dashboard's alert and recommendation rules formalisation using the current condition and the thirty-minute forecast is shown in Table 1.

Current condition	*30-minute forecast condition*	*Dashboard output*	*Recommendation type*	*Example action*
Acceptable zone	*Acceptable zone*	*Monitoring only*	*No intervention required*	*Continue normal monitoring*
Acceptable zone	*Warning zone*	*Warning alert*	*Preventive operational check*	*Inspect airflow, reduce door-opening frequency*
Warning zone	*Acceptable or warning zone*	*Warning alert*	*Operational check*	*Check local ventilation and nearby disturbance sources*
Warning zone	*High-risk zone*	*High-severity alert*	*Immediate mitigation*	*Close doors, improve ventilation, and reduce moisture ingress*
High-risk zone	*Any condition*	*High-severity alert*	*Immediate mitigation*	*Act immediately to stabilise the area*
Any condition	*High-risk zone*	*High-severity alert*	*Immediate mitigation*	*Act before the predicted excursion occurs*

Table 1. Rule-based recommendation decision matrix using current and forecast conditions

When the disturbance is consistent, showing a pattern, and suggests a likely operational cause, such as opening a door or poor ventilation, the recommendation messages can then be seen prioritising that cause on the dashboard (Zhang, 2025; Carnier, 2025).

The method for generating recommendations is based on rules derived from current reading and forecast evolutions. Therefore, when the current state and the 30-minute forecast are both within acceptable limits, the dashboard will generate an "Only normal" (monitoring only) report. If the

current state falls into a warning zone or a forecast indicates an impending excursion, the system will generate a warning alert and make a recommendation to perform an operational check (i.e., inspect airflow, reduce the frequency of opening doors, etc.). If the current state or forecast falls into a higher risk zone, the system will generate a higher-severity alert and make a recommendation for immediate mitigation (e.g., close doors, increase ventilation, reduce moisture ingress, etc.) (Parkhi, 2022; Lao, 2011).

Implementation and Testing

The implementation of the solution used the same subsystem boundaries as those defined in the design of the solution: firmware, gateway, synthetic pod, storage, forecasting, and dashboard. This modular approach also helped to reduce the integration risks and allowed for the independent validation of each component before being integrated into the overall pipeline.

A hardware Pod was developed utilising CircuitPython as a BLE peripheral to allow for periodic (i.e., at fixed intervals) reading of the SHT45 and publishing of telemetry based upon that telemetry reading. During development, the firmware must be validated, as stale firmware versions can produce misleading test results even though the gateway logic is correct. Therefore, using the firmware deployment as an integral part of the test process contributes to increased confidence in the embedded results.

The prototype was built using Python, with the gateway being the operational centre of all communications (e.g., receiving real-time telemetry data via Bluetooth Low Energy from the physical pod and receiving synthetic pod data via Transmission Control Protocol), authenticating data, monitoring link activity, and storing systems' states within SQLite. This showed that the prototype can support multiple types of data sources in a single pipeline instead of creating separate systems to support each type of pod.

The gateway created not only a simple way to ingest records; it also created a way to ensure reliability and data quality within the context of the project. For instance, the gateway performed validation of all incoming records, and it tracked duplicate records and sequence gaps; it was able to issue requests to resend records; and it also maintained link statistics (per pod) for all of the accepted records. The accepted records, along with session-aware identifiers, were written to SQLite so that when a session had been restarted, it could be differentiated from a true duplicate. The implementation also preserved errors such as missing values, out of range values, sequence gaps, duplicates and timing anomalies, instead of discarding the information silently. All of this made the prototype more suitable for being monitored to a traceable amount and eventually evaluated for forecast purposes under conditions of imperfect communication (Mahmud, 2024).

The synthetic telematics pod was utilised to add additional telemetry and generate more difficult testing conditions through simulated day-to-night patterns, seasonal patterns, and warehouse-like data buffering and intermittent communications failures. All these factors made it possible to generate a more representative exercise for ingestion, storage, and forecast generation than integrating a static data stream alone (Aloe and Maio, 2025).

Five distinct phases that verified the independent validation of the physical pod and BLE link were tested for the following steps: Verified Gateway Ingestion End To End, along with confirming Database Writes; Concurrently, Synthetic Pod And Physical Pod Testing; Validated Dashboard And Forecasting Reads From Stored Data; and Definitively Verified Live Readings Associated With Alerts and Predictions. The purpose was to confirm the correctness of both subsystems and the end-to-end integration of the full pipeline.

Table 2 provides a summary of a subset of tests run, where the rest of the tests are shown in Table 4 in the appendix, on the representative elements that comprise the full automated testing suite for the dashboard, gateway, ML, and synthetic pod subsystems. At the point in time when the overall technical project was officially created with the verification of the automated test status as indicated

in the project documentation, there were 87/87 tests were reported as passing for the project, made up of 21 dashboard tests (21/21), 48 gateway tests (48/48), 7 ML tests (7/7), and 11 synthetic pod tests (11/11). These data support the conclusion that the prototype meets its main use requirements as they relate to the ingestion, validation, storage, presentation, alerting and execution of forecast use; however, data is mainly providing evidence of and confirming that the prototype is functioning correctly and reliably under controlled prototype testing conditions rather than proving that the system is suitable for production operation in a long term warehouse environment.

These automated test results demonstrate strong functional correctness across the implemented software components, but they do not by themselves establish forecasting validity under real warehouse environmental conditions.

Testcase id	Setup/input	Expected result	Actual result	Outcome pass/fail
1	test_alerts.AlertsThresholdTests.test_combined_conditions_choose_max_severity — Combined conditions choose max severity	The assertion conditions in the test should pass.	Assertions passed.	PASS
2	test_alerts.AlertsThresholdTests.test_trajectory_uses_worst_predicted_level — Trajectory uses worst predicted level	The assertion conditions in the test should pass.	Assertions passed.	PASS
3	test_data_path.DashboardDataPathTests.test_health_context_prefers_sqlite_link_data — Health context prefers sqlite link data	The assertion conditions in the test should pass.	Assertions passed.	PASS
4	test_data_path.DashboardDataPathTests.test_latest_reading_prefers_sqlite_live_data_over_stale_csv — Latest reading prefers sqlite live data over stale csv	The assertion conditions in the test should pass.	Assertions passed.	PASS
5	test_routes_smoke.DashboardRoutesSmokeTests.test_core_routes_return_200 — Core routes return 200	The assertion conditions in the test should pass.	Assertions passed.	PASS

6	test_routes_smoke.DashboardRoutesSmokeTests.test_pages_render_expected_text — Pages render expected text	The requested output should be returned and contain the expected content.	Assertions passed.	PASS
7	test_decoder.DecoderTests.test_decode_valid_telemetry_payload — Decode valid telemetry payload	The assertion conditions in the test should pass.	Assertions passed.	PASS
8	test_forecast_runner.ForecastRunnerTests.test_csv_runner_writes_forecast_evaluation_and_case_base — Csv runner writes forecast evaluation and case base	Forecasting logic should produce the expected scenario or metrics.	Assertions passed.	PASS

9	test_gateway_interval.GatewayCommandTests.test_gateway_enforces_sixty_second_interval_when_pod_reports_ten — Gateway enforces sixty second interval when pod reports ten	The assertion conditions in the test should pass.	Assertions passed.	PASS
10	test_multi_router.MultiRouterTests.test_router_writes_per_pod_files_and_requests_resend_on_gap — Router writes per pod files and requests resend on gap	The assertion conditions in the test should pass.	Assertions passed.	PASS
11	test_preprocess_outputs.PreprocessOutputTests.test_preprocess_creates_expected_schema_and_missing_markers — Preprocess creates expected schema and missing markers	Required output or schema should be created successfully.	Assertions passed.	PASS
12	test_reassembler.JsonReassemblerTests.test_reassembles_fragmented_payload — Reassembles fragmented payload	The assertion conditions in the test should pass.	Assertions passed.	PASS

13	test_sqlite_storage.SqliteStorageTests.test_insert_or_ignore_dedupes_same_pod_and_seq — Insert or ignore dedupes same pod and seq	Duplicate input should be rejected or suppressed correctly.	Assertions passed.	PASS
14	test_sqlite_storage.SqliteStorageTests.test_schema_creation_creates_required_tables — Schema creation creates required tables	Required output or schema should be created successfully.	Assertions passed.	PASS
15	test_tcp_ingester.TcpIngesterTests.test_tcp_ingester_requests_resend_after_corrupt_line — Tcp ingester requests resend after corrupt line	The assertion conditions in the test should pass.	Assertions passed.	PASS
16	test_watchdog.WatchdogTests.test_watchdog_forces_resubscribe_after_stall — Watchdog forces resubscribe after stall	The assertion conditions in the test should pass.	Assertions passed.	PASS
17	test_event_detection.EventDetectionTests.test_recent_rh_spike_is_detected_as_door_open_like — Recent rh spike is detected as door open like	The assertion conditions in the test should pass.	Assertions passed.	PASS
18	test_filtering.FilteringTests.test_baseline_filter_reduces_spike_influence — Baseline filter reduces spike influence	The assertion conditions in the test should pass.	Assertions passed.	PASS
19	test_knn_forecast.KnnForecastTests.test_knn_forecast_returns_horizon_arrays_and_percentiles — Knn forecast returns horizon arrays and percentiles	The requested output should be returned and contain the expected content.	Assertions passed.	PASS

Table 2. *Main testcases*

Project Evaluation

Overall, the greatest asset of this project is that it has built its intended workflow based on an integrated prototype (i.e., pod, gateway, storage layer, dashboard, and forecasting logic) as opposed to separately tested proofs-of-concept. By operating within a single data path as integrated components, the project has produced an evaluation of an operationally sound monitoring system rather than just a collection of technical features.

Separation of responsibilities by means of the four-layered architecture improved clarity for testing of each subsystem (da Costa, 2022). The synthetic pod works as a good tool for verifying software integration between examples of concurrency (i.e. Ingestion at the same time), sequence processing, connection (re)behaviour and future execution during disturbed conditions or telemetry. However, the use of the synthetic pod does not replace the need for authentic variability due to different geographic warehouse locations. Therefore, while the use of the synthetic pod provides confidence in the expected behavioural performance of the software pipeline, it does not provide validation of the environmental truth or the generalisability of the forecast component within a real-world warehouse (Zhang, 2025).

Furthermore, the prototype has been enhanced with greater utility by a combination of its dashboard and recommendation functionality. The current dashboard includes not only real-time telemetry data but also other relevant information such as alerts, forecast output, and suggested course of action. This is critical because the operational user needs to be able to make a timely decision based on the latest telemetry rather than just received raw telemetry (i.e., received through long-deferred calculations) (Lao, 2011; Mohd, 2022).

While the literature has discussed BLE mesh and fully distributed clock synchronisation as potential themes for scalability and integrity, the prototype developed in this research does not claim to achieve either of those capabilities in full (Mahmud, 2024; Li, 2015). Rather, with respect to BLE (used to ingest sensor data from the physical pod) and TCP (used to ingest sensor data from the synthetic pod), support for timing/diagnostics processing is provided at the gateway end of the system and includes session-aware time alignment, anomaly detection, and retransmission logic.

Forecasting is restricted because of being limited to domain-specific warehouse data. To have a reproducible evaluation method despite that limitation, the forecast back-test was conducted on two shared pod dates that had the greatest amount of common data in the SQLite dataset, that being 29 March 2026. This resulted in finding 529 single-minute data points over 11.633 hours. The evaluation was run using a 180-minute historical window, a 30-minute forecast, and a 10-minute evaluation. In total, 76 windows were evaluated, of which 67 were stable, and 9 were disturbed. The forecast, baseline, and latency results are compiled in Table 3.

Quantitative Forecast and Latency Results

As in Table 3, the forecast error was higher when the event detector indicated that the recent history was disturbed instead of stable. The MAE for temperature increased from 0.300 °C (stable windows) to 0.442 °C (disturbed windows), and the MAE for relative humidity increased from 2.046% to 2.840%. The results indicate that the current three-hour input and thirty-minute forecasting design is more reliable when the behaviour of the environment is changing gradually, compared with when there are sudden operational disturbances (e.g., door opening or ventilation-like events) that are contributing

more to the degradation of forecast accuracy than gradual environmental changes (Zhang, 2025; Kothamali, 2025). The dew point evaluations showed the same overall pattern, with total MAEs of 0.491 °C and total RMSE of 0.657 °C.

Result set	Scenario / metric	Implemented result	Comparator/reference	Interpretation
Forecast back-test	Stable (67 windows)	Temp MAE = 0.300 °C; Temp RMSE = 0.391 °C; RH MAE = 2.046%; RH RMSE = 3.062%	Persistence: Temp MAE = 0.152 °C; Temp RMSE = 0.235 °C; RH MAE = 0.876%; RH RMSE = 1.541%	Stable windows were easier than disturbed windows, but persistence remained numerically better
Forecast back-test	Disturbed (9 windows)	Temp MAE = 0.442 °C; Temp RMSE = 0.503 °C; RH MAE = 2.840%; RH RMSE = 3.446%	Persistence: Temp MAE = 0.196 °C; Temp RMSE = 0.261 °C; RH MAE = 0.945%; RH RMSE = 1.381%	Abrupt disturbances increased error substantially
Forecast back-test	Overall (76 windows)	Temp MAE = 0.316 °C; Temp RMSE = 0.405 °C; RH MAE = 2.140%; RH RMSE = 3.110%	Persistence: Temp MAE = 0.157 °C; Temp RMSE = 0.238 °C; RH MAE = 0.884%; RH RMSE = 1.523%	The implemented forecast did not beat persistence on the selected day
Derived metric	Dew point overall	MAE = 0.491 °C; RMSE = 0.657 °C	—	Dew point followed the same overall trend
Latency	Median source-to-gateway acceptance	0.981 ms	50 samples	Very low ingestion delay
Latency	Median source-to-database visibility	18.141 ms	50 samples	Storage remained fast
Latency	Median source-to-dashboard/API visibility	48.142 ms	50 samples	Appropriate for monitoring and alerting
Latency	Worst observed end-to-end latency	88.670 ms	50 samples	Still below 0.1 s on the synthetic TCP path

Table 3. Quantitative forecast, baseline, and latency results

The examination of the baseline comparison is made explicit in Table 3; the implemented forecast method is not devoid of a simple persistence baseline for either overall performance or within either the stable or disturbed subsets on the day for which evaluations were made (Kempelis, 2023). For the period studied, the temperature MAE of the implemented forecasting method was 0.316 °C and the relative humidity MAE was 2.140%, compared to a temperature MAE of 0.157 °C and a relative humidity MAE of 0.884% for the persistence baseline; thus, currently it should be regarded as an integrated and disturbance-aware forecasting prototype instead of being regarded numerically as a stronger next-step predictor than a simple naive baseline. The usefulness of this evaluation is that it indicates where the current method has the greatest weaknesses and where future refinements to the forecasting model and/or additional warehouse data should be concentrated (Kempelis, 2023).

Along with the accuracy of a projection, Table 3 provides compact timing evidence to indicate the operational responsiveness of the test datastream prototype. The measured average latency for the synthetic TCP path measured at the time of sample creation to gateway acceptance was 0.981 ms, from gateway acceptance to database visibility was 18.141 ms, and from database visibility to dashboard / API visibility was 48.142 ms, with the greatest observed performance of 88.680 ms over the 50-sample period. This indicates that for the warehouse monitoring prototype that is being developed to manage alerting, review and short-horizon intervention rather than millisecond control, the timeliness of the prototype gateway/storage/dashboard stack is adequate for practical use of the prototype (Lao, 2011; Mohd, 2022). It should be noted that these latency results apply only to the synthetic TCP measurement path and should not be interpreted in relation to the physical BLE measurement path.

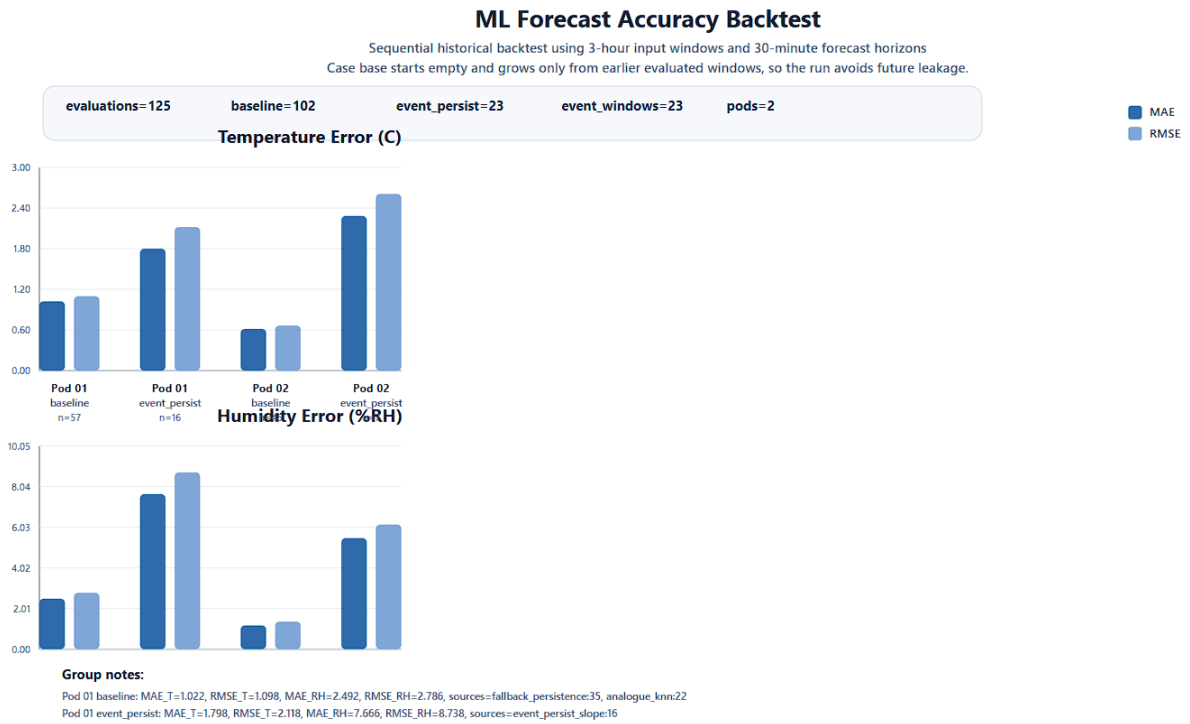


Figure 11. *Back test forecast accuracy using MAE and RMSE across pods and forecast scenarios*

Both capability and limitation are evidenced in all interpretations of Figure 11. Capability is evidenced by the ability to produce post-horizon forecast evaluation, including MAE and RMSE, and retain each

per pod/scenario, thereby closing the loop between forecast generation and subsequent review (Kempelis, 2023; FSSC, 2025). Sadly, when viewed in conjunction with Table 3, the ability to produce stable windows is materially greater than that of producing disturbed windows, and the presently implemented method performed no better than persisting with the forecast of the selected shared day. Therefore, the present forecast component can be characterized as an early-warning tool and post-hoc review component within an integrated monitoring system rather than as a fully validated industrial forecasting system; thus, the prototype is also valuable in that it provides visibility into forecast error; supports retrospective evaluation using stored data; and identifies specific areas of needed refinement within the warehouse data and forecast model (Voelker, 2020; Lao, 2011; Kempelis, 2023).

Further Work and Conclusion

This project provided an integrated Intelligent IoT prototype to monitor warehouse environmental parameters and assess short-term spoilage risk associated with spice stock. The resulting integrated system consists of the following: BLE sensors, gateway processing, SQLite database local live data storage, Flask-based Dashboard application and short-horizon forecasting sub-system providing alerts/recommendation outputs over the next 30 minutes.

Contribution from the Project is not representative of a complete industrial forecast system; instead, it is an operational end-to-end prototype developed under real-world conditions. Specifically, the sensing pods and gateway-based analytics were used to distribute monitoring through a lightweight prototype architecture.

To expand the capabilities of this system, the following actions should be undertaken: deployment of more physical pods in many different warehouse locations, to provide a greater spatial diversity and larger set of data to evaluate the system; enhancement of replay and control logic; improvement of the recommendation rules; and increased validation over a longer period of time, of the systems. The current version of the prototype provides a strong technical basis for monitoring storage conditions and forecasting short-term environmental risk in spice warehouses.

Glossary

Microclimate

The localised environment within a warehouse is different from the surrounding area, often because of airflow, doors, or stock placement.

Sensor pod

A sensing device that is used to measure the environment within a warehouse, including temperature and relative humidity levels.

Synthetic pod

A software simulation pod that is used to simulate a second environment within a warehouse, often for multiple pod testing purposes.

Gateway

The system that receives telemetry data from the pods, validates the data, and then forwards the validated data to downstream applications, such as forecasting and the dashboard.

Telemetry

The sensor data that is sent to the gateway includes any relevant metadata for the sensor readings and the environment readings.

Forecasting

The process of utilising recent telemetry history to predict future warehouse environment, including temperature and relative humidity levels, for 30 minutes into the future.

Alert

The system-generated warning is sent to indicate that the environment may need to be addressed, including the current or predicted environment levels.

Recommendation

The system-generated operational suggestions are made to respond to the environment, including the predicted environment levels.

Link quality

The measurement of the quality and reliability of the communication link between the pod and the gateway, including any issues that may be experienced, such as missing or duplicate samples, and reconnects.

Historical dataset

The stored telemetry and any relevant output are used for later review, evaluation, and improvement of the forecasting performance and output.

Dashboard

The web-based interface is used to visualise the environment, including any relevant output and forecasts, and to allow for evaluation and review.

Environmental drift

The change that is experienced in the environment within a warehouse over time is often because of changes to the operation or the weather.

Table of Abbreviations

BLE — Bluetooth Low Energy

CBR — Case-Based Reasoning

CSV — Comma-Separated Values

FSMS — Food Safety Management System

GATT — Generic Attribute Profile

IoT — Internet of Things

JSON — JavaScript Object Notation

MAE — Mean Absolute Error

ML — Machine Learning

MLOps — Machine Learning Operations

MSE — Mean Squared Error

QC — Quality Control

RH — Relative Humidity

RMSE — Root Mean Squared Error

RSSI — Received Signal Strength Indicator

TCP — Transmission Control Protocol

UI — User Interface

WAL — Write-Ahead Logging

LoRa — Low-Range Radio

References / Bibliography

Adafruit Industries (2017) Adafruit Sensirion SHT45 Precision Temperature & Humidity Sensor [online]. Available from: https://www.adafruit.com/product/5665 [Accessed 11 November 2025].

Adafruit Industries (2019) Adafruit Feather nRF52840 Express [online]. Available from: https://www.adafruit.com/product/4062 [Accessed 11 November 2025].

Adafruit Industries (no date) Adafruit IO [online]. Available from: https://io.adafruit.com [Accessed 11 November 2025].

Aloe, C.M. and Maio, A.D. (2025) Balancing Temperature and Humidity Control in Storage Location Assignment: An Optimization Perspective in Refrigerated Warehouses. *Sustainability* [online]. 17 (16), p. 7477. Available from: https://www.mdpi.com/2071-1050/17/16/7477 [Accessed 27 March 2026].

Carnier, R.M., Lahesoo, L. and Fukuda, K. (2025) Binary Anomaly Detection in Streaming IoT Traffic under Concept Drift [online]. arXiv. Available from: https://arxiv.org/abs/2510.27304 [Accessed 3 January 2026].

Components101 (2021) DHT11 Sensor Pinout, Features, Equivalents & Datasheet [online]. Available from: https://components101.com/sensors/dht11-temperature-sensor [Accessed 28 October 2025].

Cristofanelli, P., Fratticioli, C., Hazan, L., Chariot, M., Couret, C., Gazetas, O., Kubistin, D., Laitinen, A., Leskinen, A., Laurila, T., Lindauer, M., Manca, G., Ramonet, M., Trisolino, P. and Steinbacher, M. (2023) Identification of spikes in continuous ground-based in situ time series of CO_2, CH_4 and CO: an extended experiment within the European ICOS Atmosphere network. *Atmospheric Measurement Techniques* [online]. 16 (24), pp. 5977–5994. Available from: https://amt.copernicus.org/articles/16/5977/2023/ [Accessed 16 February 2026].

da Costa, T.P., Gillespie, J., Cama-Moncunill, X., Ward, S., Condell, J., Ramanathan, R. and Murphy, F. (2022) A Systematic Review of Real-Time Monitoring Technologies and Its Potential Application to Reduce Food Loss and Waste: Key Elements of Food Supply Chains and IoT Technologies. *Sustainability* [online]. 15 (1), p. 614. Available from: https://www.mdpi.com/2071-1050/15/1/614 [Accessed 29 Novemeber 2025].

Eltholth, A.A. (2023) Improved Spectrum Coexistence in 2.4 GHz ISM Band Using Optimized Chaotic Frequency Hopping for Wi-Fi and Bluetooth Signals. *Sensors* [online]. 23 (11), p. 5183. Available from: https://www.mdpi.com/1424-8220/23/11/5183 [Accessed 5 December 2025].

Espressif Systems (no date) ESP8266 Overview [online]. Available from: https://www.espressif.com/en/products/socs/esp8266 [Accessed 28 October 2025].

FSSC (2025) Version 6 Documents [online]. Available from: https://www.fssc.com/fssc-22000/documents/version-6-documents/ [Accessed 14 January 2026].

Górnicki, K., Winiczenko, R., Kaleta, A. and Chońska, A. (2017) Evaluation of models for the dew point temperature determination. *Technical Sciences* [online]. 20 (3), pp. 241–257. Available from: https://czasopisma.uwm.edu.pl/index.php/ts/article/view/5425 [Accessed 20 November 2025].

Gowrishankar, V., Veena, P., Ponmurugan, P., Annapoorani, B.T., Vijayakumar, P. and Siva Ramkumar, M. (2023) Edge Computing Enabled Smart Warehouse Management System for Food Processing Industries. In: *2023 14th International Conference on Computing Communication and Networking*

Technologies (ICCCNT) [online]. Available from: https://ieeexplore.ieee.org/document/10308398/ [Accessed 21 November 2025].

Hatem, A.S. and Abed, J.K. (2022) Design of Patient Health Monitoring Using ESP8266 and Adafruit IO Dashboard. In: *2022 International Conference on Electrical, Computer and Energy Technologies (ICECET)* [online]. Available from: https://ieeexplore.ieee.org/document/9872787 [Accessed 14 January 2026].

Irvine, J. (2016) Comparison of Power Consumption of Wi-Fi Inbuilt Internet of Things Device with Bluetooth Low Energy [online]. Available from: https://www.academia.edu/download/100577118/10005628.pdf [Accessed 23 December 2025].

Jablonski, A. and Mendrok, K. (2025) Automatic threshold setting for anomaly detection. *Mechanical Systems and Signal Processing* [online]. 228, p. 112462. Available from: https://www.sciencedirect.com/science/article/pii/S0888327025001633 [Accessed 14 February 2026].

Karam, L., Salloum, T., El Hage, R., Hassan, H. and Hassan, H.F. (2021) How can packaging, source and food safety management system affect the microbiological quality of spices and dried herbs? The case of a developing country. *International Journal of Food Microbiology* [online]. 353, p. 109295. Available from: https://www.sciencedirect.com/science/article/abs/pii/S0168160521002543?via%3Dihub [Accessed 3 November 2025].

Kempelis, A., Narigina, M., Osadcijs, E., Patlins, A. and Romanovs, A. (2023) Machine Learning-based Sensor Data Forecasting for Precision Evaluation of Environmental Sensing. In: *2023 IEEE 10th Jubilee Workshop on Advances in Information, Electronic and Electrical Engineering (AIEEE)* [online]. pp. 1–6. Available from: https://ieeexplore.ieee.org/abstract/document/10135031 [Accessed 2 January 2026].

Khattach, O., Moussaoui, O. and Hassine, M. (2025) End-to-End Architecture for Real-Time IoT Analytics and Predictive Maintenance Using Stream Processing and ML Pipelines. *Sensors* [online]. 25 (9), p. 2945. Available from: https://www.mdpi.com/1424-8220/25/9/2945 [Accessed 27 February 2026].

Kim, M. and Paek, J. (2025) Multi-Connection Scheduling with Fair Resource Management for Scalable Bluetooth Low Energy Networks. *IEEE Internet of Things Journal* [online]. pp. 1–1. Available from: https://ieeexplore.ieee.org/abstract/document/11275955 [Accessed 4 January 2026].

Kothamali, P.R. and Banik, S. (2025) Enhancing Edge Computing Security with AI-Driven Threat Detection and Preventative Measures. *Journal of Computing Innovations and Applications* [online]. 3 (1), pp. 8–21. Available from: https://ciajournal.com/index.php/jcia/article/view/10 [Accessed 23 February 2026].

Kulanuwat, L., Chantrapornchai, C., Maleewong, M., Wongchaisuwat, P., Wimala, S., Sarinnapakorn, K. and Boonya-aroonnet, S. (2021) Anomaly Detection Using a Sliding Window Technique and Data Imputation with Machine Learning for Hydrological Time Series. *Water* [online]. 13 (13), p. 1862. Available from: https://www.mdpi.com/2073-4441/13/13/1862 [Accessed 15 March 2026].

Lao, S.I., Choy, K.L., Ho, G.T.S., Tsim, Y.C. and Lee, C.K.H. (2011) Real-time inbound decision support system for enhancing the performance of a food warehouse. *Journal of Manufacturing Technology Management* [online]. 22 (8), pp. 1014–1031. Available from: https://www.emerald.com/jmtm/article-abstract/22/8/1014/440376/Real-time-inbound-decision-support-system-for?redirectedFrom=fulltext [Accessed 14 December 2025].

Li, J., Mechitov, K.A., Kim, R.E. and Spencer, B.F. (2015) Efficient time synchronization for structural health monitoring using wireless smart sensor networks. *Structural Control and Health Monitoring* [online]. 23 (3), pp. 470–486. Available from: https://onlinelibrary.wiley.com/doi/10.1002/stc.1782 [Accessed 4 December 2025].

Martin, R.C. (2012) The Clean Architecture [online]. Available from: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html [Accessed 2 February 2026].

Mohamed Mahmud, A. (2024) Development and optimization of a Bluetooth Low Energy (BLE) Mesh network for environmental monitoring and alerting [Bachelor's thesis]. Vaasan ammattikorkeakoulu [online]. Available from: https://www.theseus.fi/handle/10024/866129 [Accessed 13 December 2025].

Murph, A.C., Gibson, G.C., Amona, E.B., Beesley, L.J., Castro, L.A., Del Valle, S.Y. and Osthus, D. (2025) Synthetic method of analogues for emerging infectious disease forecasting. *PLOS Computational Biology* [online]. 21 (6), p. e1013203. Available from: https://pmc.ncbi.nlm.nih.gov/articles/PMC12303386/ [Accessed 17 February 2026].

Navarro, S. and Navarro, H. (2018) The significance of vapor pressure in quality preservation of stored commodities under gastight conditions. *Julius-Kühn-Archiv* [online]. (463), pp. 549–556. Available from: https://www.academia.edu/download/82973895/9803.pdf [Accessed 16 December 2025].

Nentwich, C. and Reinhart, G. (2021) A Combined Anomaly and Trend Detection System for Industrial Robot Gear Condition Monitoring. *Applied Sciences* [online]. 11 (21), p. 10403. Available from: https://www.mdpi.com/2076-3417/11/21/10403 [Accessed 24 March 2026].

Parkhi, V., Chavhan, N., Chandak, S., Chaware, B. and Bongarde, P. (2023) An IoT Based Environment Monitoring & Controlling System for Food Grain Warehouse. In: Joby, P.P., Balas, V.E. and Palanisamy, R. (eds.) *IoT Based Control Networks and Intelligent Systems* [online]. Singapore: Springer, pp. 217–227. Available from: https://link.springer.com/chapter/10.1007/978-981-19-5845-8_16 [Accessed 13 November 2025].

Peddoju, S.K. and Upadhyay, H. (2020) Evaluation of IoT Data Visualization Tools and Techniques. In: Anouncia, S., Gohel, H. and Vairamuthu, S. (eds.) *Data Visualization* [online]. Singapore: Springer, pp. 115–139. Available from: https://link.springer.com/chapter/10.1007/978-981-15-2282-6_7 [Accessed 30 December 2025].

Process Parameters Ltd. (2025) PRT Air Temperature Sensor (PPL9-P) [online]. Available from: https://www.processparameters.co.uk/pt100-temperature-sensors/prt-air-temperature-sensor-ppl9-p/ [Accessed 1 April 2026].

Ramires, M., Campaniço, A., Meneses, F. and Moreira, A. (2024) Assessing BLE RSSI Fingerprinting for Product Tracking in a Warehouse [online]. pp. 1–6. Available from: https://ieeexplore.ieee.org/document/10817464 [Accessed 2 December 2025].

Shaiful, M.S.S., Ambar, R., Abd Wahab, M.H. and Abd Jamil, M.M. (2022) IoT-based Data Monitoring and Environment Controlling System for Oyster Mushroom Mini-farm. In: *2022 International Visualization, Informatics and Technology Conference (IVIT)* [online]. pp. 309–316. Available from: https://ieeexplore.ieee.org/document/10033410/ [Accessed 2 March 2026].

SQLite.org (no date) Appropriate Uses For SQLite [online]. Available from: https://sqlite.org/whentouse.html [Accessed 27 March 2026].

SQLite.org (2025) SQLite User Forum: Multiple Writers [online]. Available from: https://sqlite.org/forum/info/b4e8b29ae409cd198652c6b7e70b53b702f269e67e1d2573d627feeba37bbf85 [Accessed 27 March 2026].

Voelker, A.L., Sommer, A.A. and Mauer, L.J. (2020) Moisture sorption behaviors, water activity-temperature relationships, and physical stability traits of spices, herbs, and seasoning blends containing crystalline and amorphous ingredients. *Food Research International* [online]. 136, p. 109608. Available from: https://www.sciencedirect.com/science/article/abs/pii/S0963996920306335?via%3Dihub [Accessed 15 November 2025].

Wang, C., Yang, X. and Xu, Y. (2020) Research on Intelligent Measurement and Control System of Warehouse Temperature and Humidity. In: *2020 2nd International Conference on Artificial Intelligence and Advanced Manufacture (AIAM)* [online]. pp. 266–270. Available from: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9426002 [Accessed 11 January 2026].

Wi-Fi Alliance, LoRa Alliance and Wireless Broadband Alliance (2019) Wi-Fi & LoRaWAN® Deployment Synergies: Expanding Addressable Use Cases for the Internet of Things [online]. Available from: https://lora-alliance.org/wp-content/uploads/2020/11/wi-fi-and-lorawanr-deployment-synergies.pdf [Accessed 12 February 2026].

Yuksel, M.E. (2020) Power Consumption Analysis of a Wi-Fi-based IoT Device. *Electrica* [online]. 20 (1), pp. 62–70. Available from: https://electricajournal.org/index.php/pub/article/view/756 [Accessed 4 December 2025].

Zhang, W., Zhang, X. and Zhou, X. (2025) IoT-Based Multi-Sensor Environmental Monitoring and Intelligent Control in Automated Warehouses Using Fuzzy Logic and Deep Learning. *Informatica* [online]. 49 (33). Available from: https://informatica.si/index.php/informatica/article/view/8257 [Accessed 7 December 2025].

Appendix A: First Appendix

Supporting documentation of the implemented systems can be found in this appendix using screenshots of the various system features to support the operation of the gateway, including but not limited to forecasting and synthetic pods. There will also be screenshots provided of the various terminal outputs during the implementation and testing discussions, as providing such terminal outputs in the main body of the report would create an excessive number of pages to accommodate.

Dashboard Overview

The main overview of the dashboard can be found in Figure 12. The dashboard is designed to provide the most recent readings from each of the pods, indicate the current state of storage, and provide a rule-based alert level to the user. The goal of this overview page is to allow the user to determine the basic current condition of the warehouse, as well as provide a summary of the most important recommended actions.

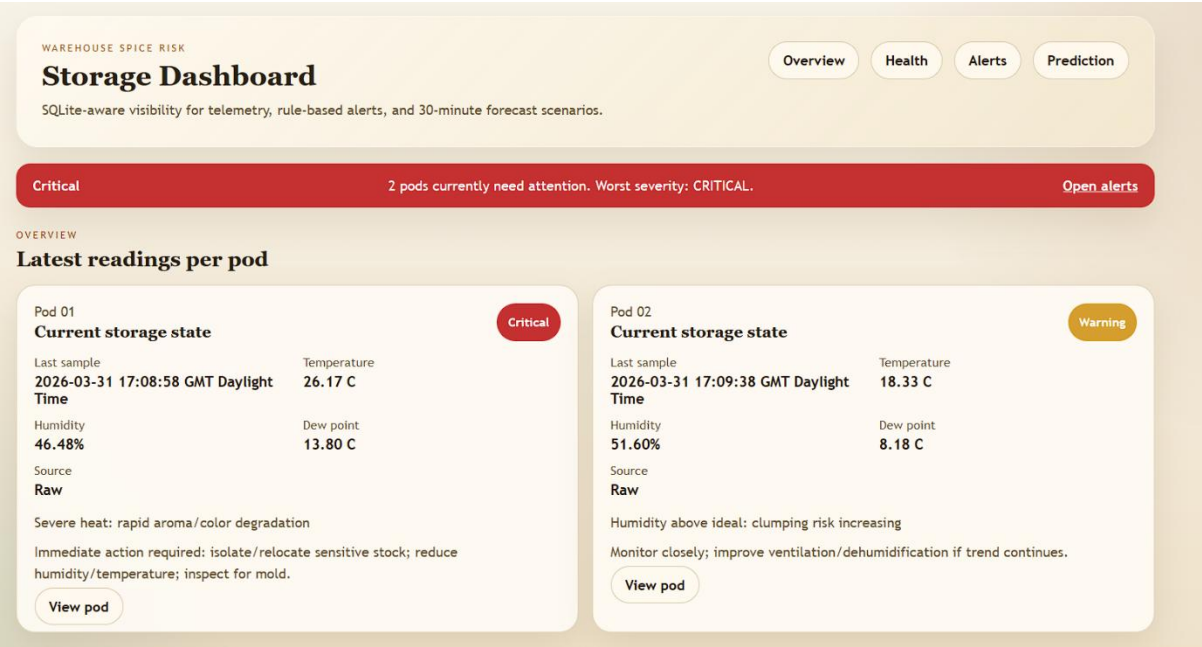


Figure 12. *Dashboard main overview page*

Health and Link Metrics Page

The health page view of the dashboard is shown in Figure 13. The health page view contains information on the health of the links to the gateway, such as the total number of received samples; missing samples, duplicate samples, reconnect events, and RSSI (if available). This information is beneficial for determining that the quality of communication between the pods and gateway is stable during operation.

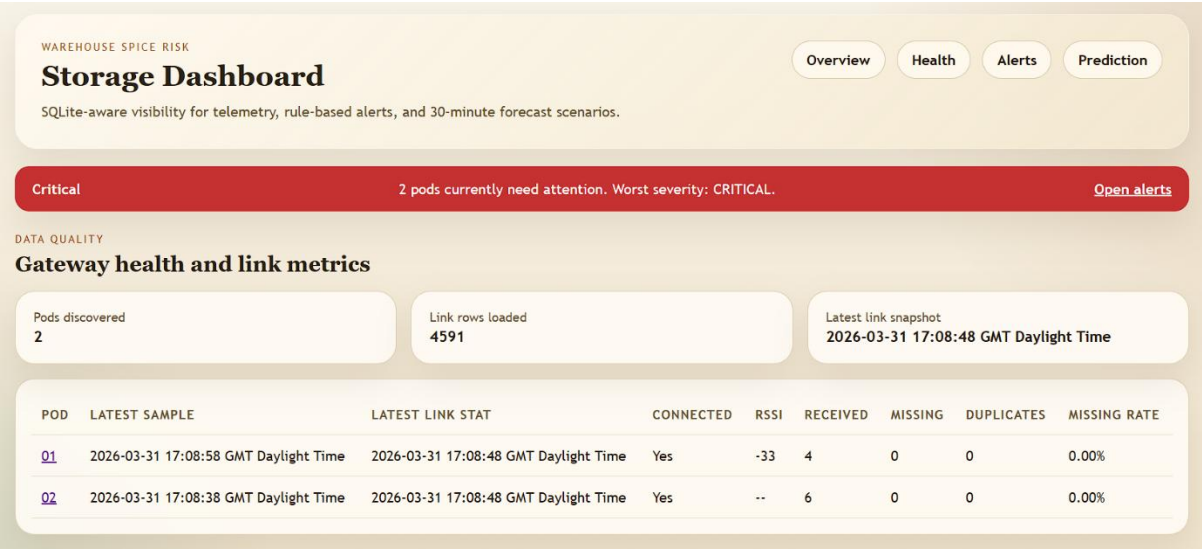


Figure 13. *Health page*

Alerts Page

Figure 14 shows the active alerts page view of the dashboard. The alerts page view shows all current storage alerts, which are sorted by severity, as well as any recommendations associated with the alerts that were generated. As well, the alert page view will show any alerts that have been temporarily acknowledged, which aids in providing short-term alert management without completely removing the event from visibility in the system.

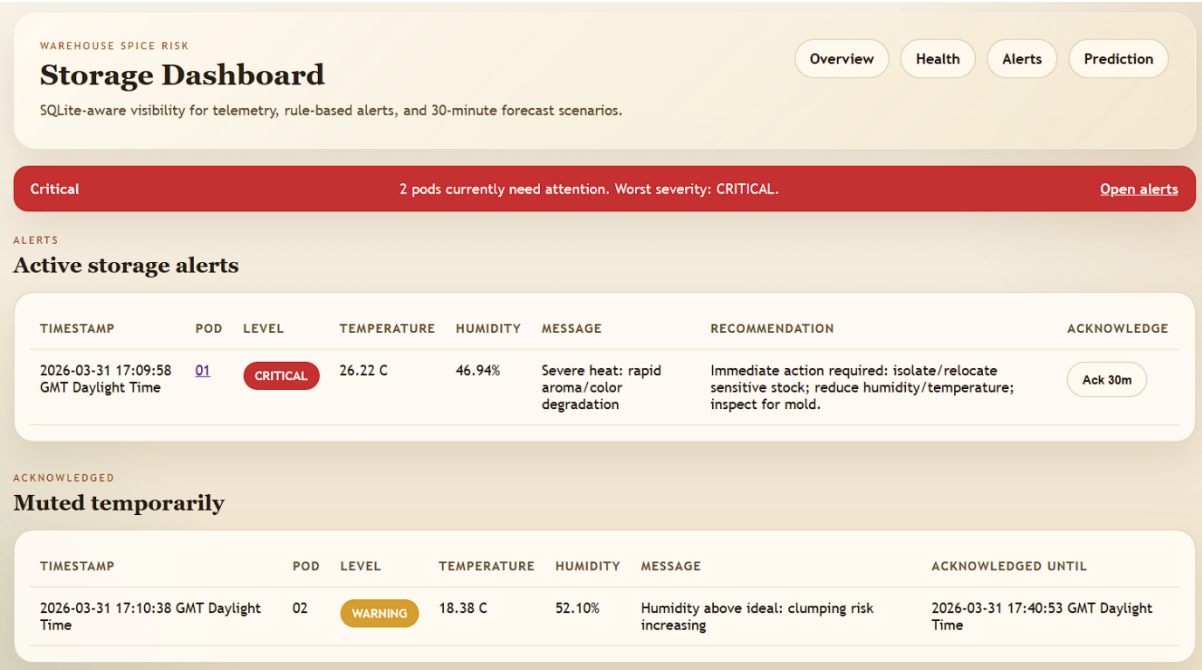


Figure 14. *Alerts page*

Readings History Page

The reading's history page view can be seen in Figure 15. The reading's history page view displays the most recent temperature, relative humidity, and dew point trends for the selected time range. By presenting this information in this manner, the user can analyse the environmental behaviour over time and not be restricted to just the most recent single reading.

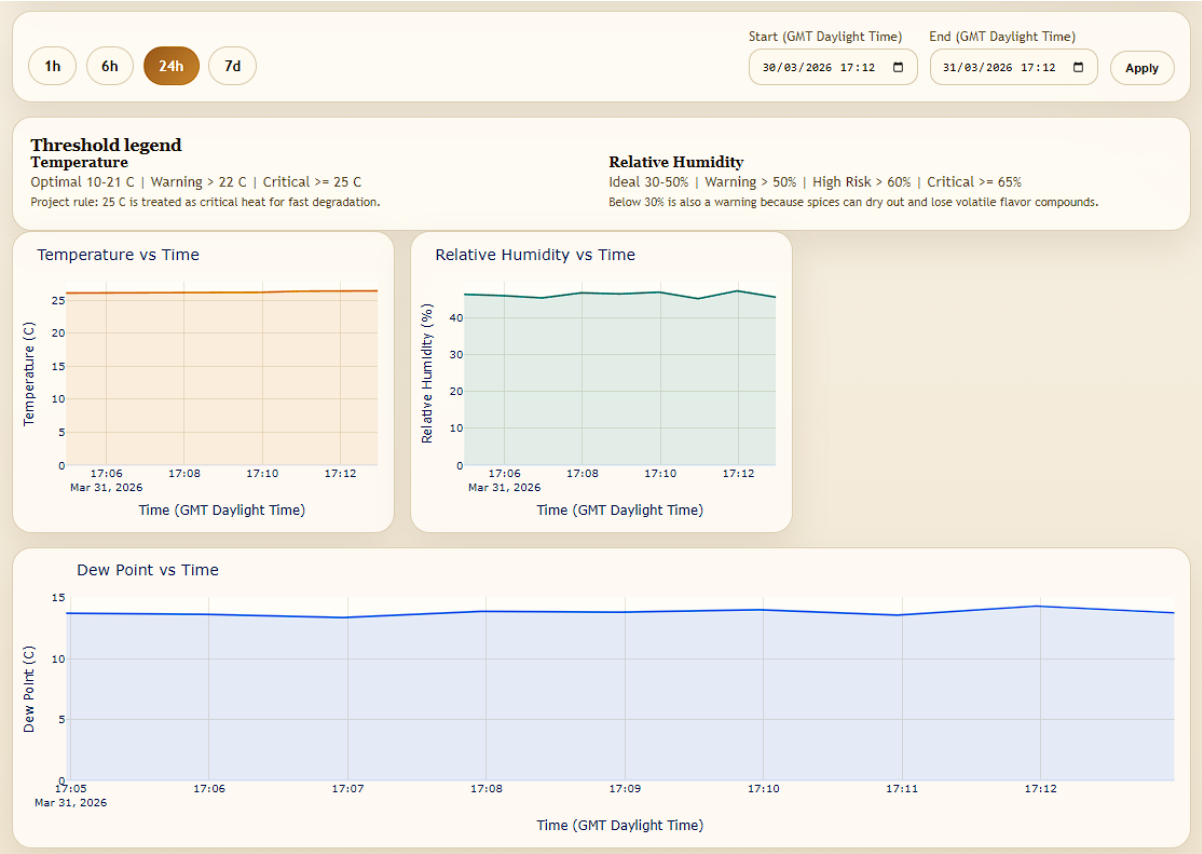
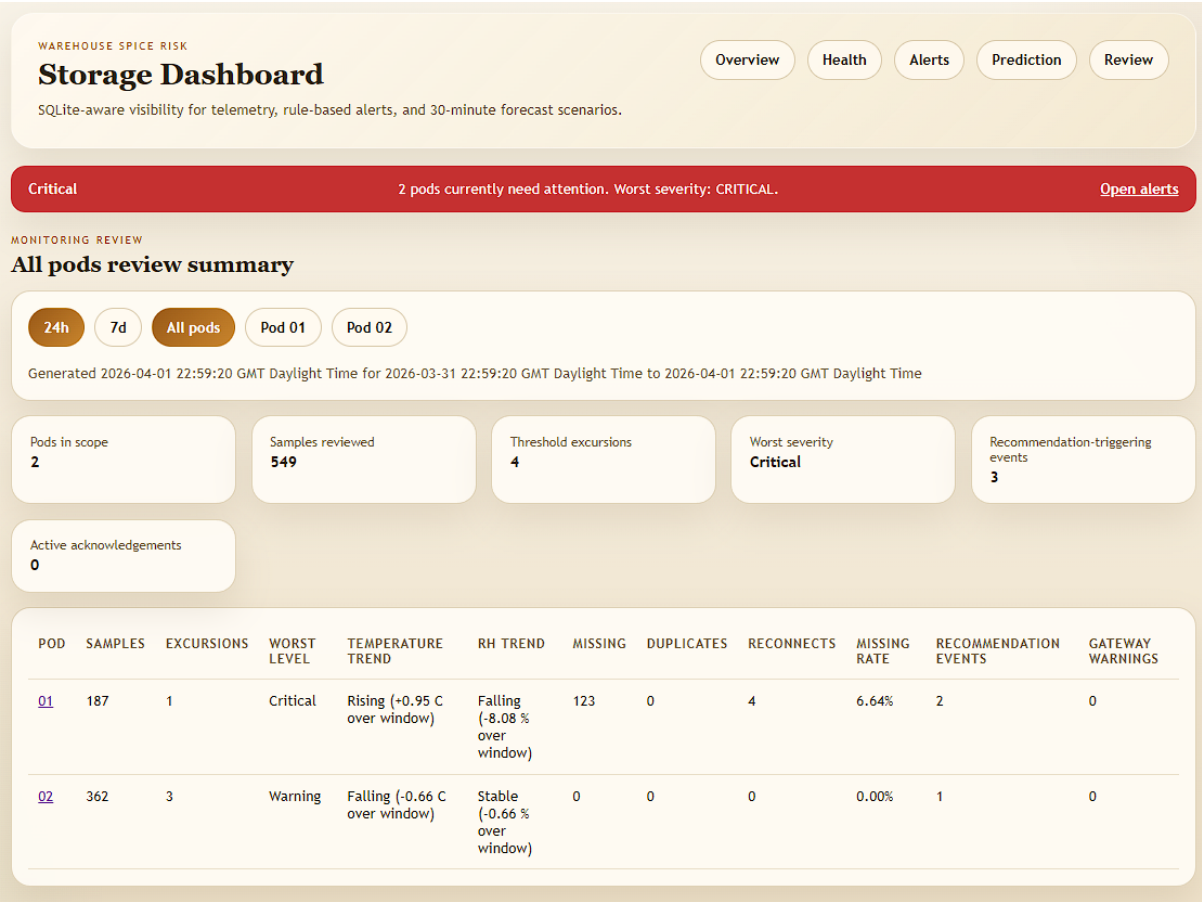


Figure 15. *Readings page*

Review Summary Page

A summary of the recent monitoring activity across all pods is displayed on the monitoring review page (Figure 16). The review page view contains data such as threshold excursions, threshold excursions that had the worst severity level, recommendation-triggering events, and the quality of the links. This information allows for periodic evaluation of the behaviour of the system over the selected time window.



POD	SAMPLES	EXCURSIONS	WORST LEVEL	TEMPERATURE TREND	RH TREND	MISSING	DUPLICATES	RECONNECTS	MISSING RATE	RECOMMENDATION EVENTS	GATEWAY WARNINGS
01	187	1	Critical	Rising (+0.95 C over window)	Falling (-8.08 % over window)	123	0	4	6.64%	2	0
02	362	3	Warning	Falling (-0.66 C over window)	Stable (-0.66 % over window)	0	0	0	0.00%	1	0

Figure 16. *Review page*

Gateway Terminal Output

Figure 17 presents the terminal output from the gateway during the operation of the multiple pods. The terminal output illustrates concurrent data ingestion from both the physical BLE pod and the imaginary or "synthetic" TCP pod; subsequently, the normal link-related statistics will relate directly to the sequence-sorted telemetry processing. The information provided will confirm that the gateway is operational and receiving live data from both pod types.

```
PS C:\Users\TERA MAX\Desktop\DSP> cd "C:\Users\TERA MAX\Desktop\DSP\src"
>> .\.venv\Scripts\python.exe -m gateway.cli.gateway_cli multi --ble-address F2:9A:41:2B:5B:55 --tcp-port 8765 --storage sqlite --db-path data/db/telemetry.sqlite --verbose
>>
2026-03-31 17:03:37,703 DEBUG gateway.multi.orchestrator: Multi-pod gateway starting with BLE prefix=SHT45-POD- tcp_port=8765 interval=60s storage=sqlite
2026-03-31 17:03:37,709 DEBUG gateway.multi.orchestrator: Primary SQLite DB: C:\Users\TERA MAX\Desktop\DSP\src\data\db\telemetry.sqlite
2026-03-31 17:03:37,737 DEBUG gateway.ingesters.tcp_ingester: Synthetic TCP listener ready on ('127.0.0.1', 8765)
2026-03-31 17:03:38,423 DEBUG gateway.ingesters.tcp_ingester: Synthetic pod connected from ('127.0.0.1', 62498)
2026-03-31 17:03:38,423 INFO gateway.multi.router: [pod=02 source=TCP] connected
2026-03-31 17:03:38,432 INFO gateway.multi.router: [pod=02 source=TCP] seq=1 temp=18.598 rh=53.876 flags=0
2026-03-31 17:03:58,439 DEBUG gateway.ble.client.F29A412B5B55: Connecting to F2:9A:41:2B:5B:55 (F2:9A:41:2B:5B:55)
2026-03-31 17:04:00,613 DEBUG gateway.ble.client.F29A412B5B55: Connected to F2:9A:41:2B:5B:55 (F2:9A:41:2B:5B:55)
2026-03-31 17:04:00,613 INFO gateway.multi.router: [pod=01 source=BLE] connected
2026-03-31 17:04:00,671 DEBUG gateway.ble.client.F29A412B5B55: status firmware_version=0.1.0-phase1 last_error=0 sample_interval_s=60
0.0000
2026-03-31 17:04:58,675 DEBUG gateway.ble.client.F29A412B5B55: telemetry pod_id=01 seq=2 ts_uptime_s=64.0 temp_c=26.11 rh_pct=46.36 flags=0 quality_flags=-
2026-03-31 17:04:58,716 INFO gateway.multi.router: [pod=01 source=BLE] seq=2 temp=26.11 rh=46.36 flags=0
2026-03-31 17:05:18,281 DEBUG gateway.multi.orchestrator: stats pod=02 source=TCP received=2 missing=0 duplicates=0 corrupt_count=0 reconnects=0 missing_rate=0.0000
2026-03-31 17:05:18,282 DEBUG gateway.multi.orchestrator: stats pod=01 source=BLE received=1 missing=0 duplicates=0 corrupt_count=0 reconnects=0 missing_rate=0.0000
2026-03-31 17:05:38,444 INFO gateway.multi.router: [pod=02 source=TCP] seq=3 temp=18.81 rh=51.014 flags=0
2026-03-31 17:05:48,298 DEBUG gateway.multi.orchestrator: stats pod=02 source=TCP received=3 missing=0 duplicates=0 corrupt_count=0 reconnects=0 missing_rate=0.0000
2026-03-31 17:05:48,299 DEBUG gateway.multi.orchestrator: stats pod=01 source=BLE received=1 missing=0 duplicates=0 corrupt_count=0 reconnects=0 missing_rate=0.0000
2026-03-31 17:05:58,734 DEBUG gateway.ble.client.F29A412B5B55: telemetry pod_id=01 seq=3 ts_uptime_s=124.0 temp_c=26.15 rh_pct=46.0 flags=0 quality_flags=-
2026-03-31 17:05:58,735 INFO gateway.multi.router: [pod=01 source=BLE] seq=3 temp=26.15 rh=46.0 flags=0
2026-03-31 17:06:18,314 DEBUG gateway.multi.orchestrator: stats pod=02 source=TCP received=3 missing=0 duplicates=0 corrupt_count=0 reconnects=0 missing_rate=0.0000
2026-03-31 17:06:18,315 DEBUG gateway.multi.orchestrator: stats pod=01 source=BLE received=2 missing=0 duplicates=0 corrupt_count=0 reconnects=0 missing_rate=0.0000
2026-03-31 17:06:38,450 INFO gateway.multi.router: [pod=02 source=TCP] seq=4 temp=18.217 rh=51.628 flags=0
2026-03-31 17:06:48,326 DEBUG gateway.multi.orchestrator: stats pod=02 source=TCP received=4 missing=0 duplicates=0 corrupt_count=0 reconnects=0 missing_rate=0.0000
2026-03-31 17:06:48,327 DEBUG gateway.multi.orchestrator: stats pod=01 source=BLE received=2 missing=0 duplicates=0 corrupt_count=0 reconnects=0 missing_rate=0.0000
2026-03-31 17:06:58,740 DEBUG gateway.ble.client.F29A412B5B55: telemetry pod_id=01 seq=4 ts_uptime_s=184.1 temp_c=26.09 rh_pct=45.39 flags=0 quality_flags=-
2026-03-31 17:06:58,755 INFO gateway.multi.router: [pod=01 source=BLE] seq=4 temp=26.09 rh=45.39 flags=0
```

Figure 17. *Gateway running in the terminal*

Forecast Runner Terminal Output

The terminal output of the forecast runner is found in Figure 18. It illustrates the execution of the 30-minute forecast cycle, including the generation of the forecast and associated recommendations, and post-horizon evaluation of the forecasting subsystem. The output will indicate that the forecasting subsystem has been operational and has produced defined measurable outputs based on the telemetry provided from the gateway.

```
>> .\.venv\Scripts\python.exe -m gateway.cli.forecast_cli run --all --every-minutes 30 --storage sqlite --db-path data/db/telemetry.sqlite
>>
2026-03-31 17:04:05,045 INFO __main__: forecast cycle start ts=2026-03-31T16:04:05.045367+00:00 pods=01,02 backend=sqlite
2026-03-31 17:04:05,056 INFO gateway.forecast.runner: Skipping evaluation for pod=01 ts=2026-03-29T18:19:00Z; horizon data still incomplete.
2026-03-31 17:04:05,063 INFO gateway.forecast.runner: Skipping evaluation for pod=02 ts=2026-03-29T18:19:00Z; horizon data still incomplete.
2026-03-31 17:04:05,067 INFO gateway.forecast.runner: Skipping evaluation for pod=01 ts=2026-03-29T21:22:00Z; horizon data still incomplete.
2026-03-31 17:04:05,076 INFO gateway.forecast.runner: evaluation pod=01 ts=2026-03-30T01:56:00Z scenario=baseline maeT=1.173 rmseT=1.173 maeRH=0.952 rmseRH=0.965
2026-03-31 17:04:05,100 INFO gateway.forecast.runner: evaluation pod=02 ts=2026-03-30T01:56:00Z scenario=baseline maeT=0.723 rmseT=0.730 maeRH=1.759 rmseRH=1.777
2026-03-31 17:04:05,136 INFO gateway.forecast.runner: forecast pod=01 ts=2026-03-30T01:59:00Z event=False type=none source=analogue_knn neighbors=7 missing_rate=0.000
2026-03-31 17:04:05,168 INFO gateway.forecast.runner: forecast pod=02 ts=2026-03-31T16:03:00Z event=False type=none source=analogue_knn neighbors=10 missing_rate=0.994
```

Figure 18. *Forecast running in the terminal.*

Synthetic Pod Terminal Output

Figure 19 presents the terminal output from the synthetic pod. The terminal circuit will illustrate that the synthetic pod produced credible simulated environmental telemetry and delivered that telemetry to the gateway, and allows for the dual-pod assessment of the system despite the limited number of physical devices available for testing.

```
PS C:\Users\TERA MAX\Desktop\DSP>
PS C:\Users\TERA MAX\Desktop\DSP> cd "C:\Users\TERA MAX\Desktop\DSP\src"
>> .\.venv\Scripts\python.exe .\synthetic_pod\pod2_sim.py --gateway-port 8765 --pod-id 02 --interval 60
>>
2026-03-31 17:03:28,335 WARNING synthetic_pod: Gateway connection failed: [WinError 1225] The remote computer refused the network connection. Retrying in 2s.
2026-03-31 17:03:32,384 WARNING synthetic_pod: Gateway connection failed: [WinError 1225] The remote computer refused the network connection. Retrying in 2s.
2026-03-31 17:03:36,402 WARNING synthetic_pod: Gateway connection failed: [WinError 1225] The remote computer refused the network connection. Retrying in 2s.
2026-03-31 17:03:38,409 INFO synthetic_pod: Connected to gateway at 127.0.0.1:8765
2026-03-31 17:03:38,410 INFO synthetic_pod: [pod=02 zone=entrance_disturbed] seq=1 temp=18.598 rh=53.876 disturbance=off event=steady active_hours=yes burst_fault=no
2026-03-31 17:04:38,424 INFO synthetic_pod: [pod=02 zone=entrance_disturbed] seq=2 temp=18.699 rh=51.512 disturbance=off event=steady active_hours=yes burst_fault=no
2026-03-31 17:05:38,440 INFO synthetic_pod: [pod=02 zone=entrance_disturbed] seq=3 temp=18.810 rh=51.014 disturbance=off event=steady active_hours=yes burst_fault=no
```

Figure 19. *Synthetic pod running in terminal*

Remaining Automated Test Cases

The remaining automated test cases, which were not included in the main report's test table, can be found in Table 4. These automated test cases are provided for the purpose of keeping a complete history of automated tests and for the purpose of providing a complete history of tests that were executed. The main report's table of test cases and the remaining automated test case table provide an overall view of the entire breadth of tests that were conducted for the dashboard, gateway, storage, forecasting, validation and synthetic pod subsystems.

Testcase id	Setup/input	Expected result	Actual result	Outcome pass/fail
20	test_alerts.AlertsThresholdTests.test_humidity_boundaries — Humidity boundaries	The assertion conditions in the test should pass.	Assertions passed.	PASS
21	test_alerts.AlertsThresholdTests.test_optimal_and_acceptable_levels — Optimal and acceptable levels	The assertion conditions in the test should pass.	Assertions passed.	PASS
22	test_alerts.AlertsThresholdTests.test_temperature_boundaries — Temperature boundaries	The assertion conditions in the test should pass.	Assertions passed.	PASS
23	test_alerts.AlertsThresholdTests.test_trajectory_keeps_first_occurrence_of_worst_level — Trajectory keeps first occurrence of worst level	The assertion conditions in the test	Assertions passed.	PASS

		should pass.		
24	test_data_path.DashboardDataPathTests.test_health_context_matches_link_rows_for_zero_padded_pod_ids — Health context matches link rows for zero-padded pod ids	The assertion conditions in the test should pass.	Assertions passed.	PASS
25	test_data_path.DashboardDataPathTests.test_latest_reading_uses_newest_sample_even_when_it_is_incomplete — Latest reading uses newest sample even when it is incomplete	The assertion conditions in the test should pass.	Assertions passed.	PASS
26	test_data_path.DashboardDataPathTests.test_raw_reader_preserves_leading_zero_pod_ids_and_append_order — Raw reader preserves leading zero pod ids and append order	The assertion conditions in the test should pass.	Assertions passed.	PASS
27	test_routes_smoke.DashboardRoutesSmokeTests.test_acknowledge_post_redirects_back_to_alerts — Acknowledge post redirects back to alerts	The assertion conditions in the test should pass.	Assertions passed.	PASS
28	test_routes_smoke.DashboardRoutesSmokeTests.test_dashboard_can_opt_in_to_auto_refresh — Dashboard can opt in to auto refresh	The assertion conditions in the test should pass.	Assertions passed.	PASS
29	test_routes_smoke.DashboardRoutesSmokeTests.test_dashboard_disables_auto_refresh_by_default — Dashboard disables auto refresh by default	The assertion conditions in the test should pass.	Assertions passed.	PASS
30	test_timezone_display.DashboardTimezoneTests.test_custom_range_input_is_interprete	The requested	Assertions passed.	PASS

	d_in_display_timezone — Custom range input is interpreted in display timezone	output should be returned and contain the expected content.		
31	test_timezone_display.DashboardTimezoneTests.test_display_format_uses_local_dashboard_timezone — Display format uses local dashboard timezone	The requested output should be returned and contain the expected content.	Assertions passed.	PASS
32	test_csv_import.CsvImportTests.test_import_csv_history_copies_canonical_and_legacy_rows_once — Import csv history copies canonical and legacy rows once	The assertion conditions in the test should pass.	Assertions passed.	PASS
33	test_decoder.DecoderTests.test_decode_rejects_missing_field — Decode rejects missing field	The assertion conditions in the test should pass.	Assertions passed.	PASS
34	test_decoder.DecoderTests.test_decode_valid_status_payload — Decode valid status payload	The assertion conditions in the test should pass.	Assertions passed.	PASS
35	test_dewpoint.DewPointTests.test_clamps_humidity_into_supported_range — Clamps humidity into supported range	Values should stay within the expected range or cadence.	Assertions passed.	PASS

36	test_dewpoint.DewPointTests.test_magnus_formula_matches_expected_value — Magnus formula matches expected value	The assertion conditions in the test should pass.	Assertions passed.	PASS
37	test_dewpoint.DewPointTests.test_returns_none_for_missing_values — Returns none for missing values	The requested output should be returned and contain the expected content.	Assertions passed.	PASS
38	test_gateway_interval.GatewayCommandTests.test_gateway_does_not_double_send_when_user_explicitly_sets_interval — Gateway does not double send when user explicitly sets interval	The assertion conditions in the test should pass.	Assertions passed.	PASS
39	test_gateway_interval.GatewayCommandTests.test_gateway_does_not_send_control_without_command_or_mismatch — Gateway does not send control without command or mismatch	The assertion conditions in the test should pass.	Assertions passed.	PASS
40	test_gateway_interval.GatewayCommandTests.test_gateway_sends_explicit_control_command_only — Gateway sends explicit control command only	The assertion conditions in the test should pass.	Assertions passed.	PASS
41	test_multi_router.MultiRouterTests.test_router_detects_small_sequence_restart_when_uptime_keeps_advancing — Router detects small sequence restart when uptime keeps advancing	The assertion conditions in the test should pass.	Assertions passed.	PASS
42	test_multi_router.MultiRouterTests.test_router_detects_soft_reload_sequence_drop_wi	The assertion	Assertions passed.	PASS

	th_higher_uptime — Router detects soft reload sequence drop with higher uptime	conditions in the test should pass.		
43	test_multi_router.MultiRouterTests.test_router_throttles_repeat_resend_requests_after_duplicate_progress — Router throttles repeat resend requests after duplicate progress	Duplicate input should be rejected or suppressed correctly.	Assertions passed.	PASS
44	test_preprocess_outputs.PreprocessOutputTests.test_export_training_concatenates_processed_rows — Export training concatenates processed rows	The assertion conditions in the test should pass.	Assertions passed.	PASS
45	test_process_lock.ProcessLockTests.test_acquire_overwrites_stale_lock_when_pid_is_reused_by_other_process — Acquire overwrites stale lock when pid is reused by other process	The assertion conditions in the test should pass.	Assertions passed.	PASS
46	test_process_lock.ProcessLockTests.test_acquire_overwrites_stale_lock_when_process_is_not_running — Acquire overwrites stale lock when process is not running	The assertion conditions in the test should pass.	Assertions passed.	PASS
47	test_process_lock.ProcessLockTests.test_active_legacy_lock_blocks_new_sqlite_file_lock — Active legacy lock blocks new sqlite file lock	The assertion conditions in the test should pass.	Assertions passed.	PASS
48	test_process_lock.ProcessLockTests.test_build_lock_path_targets_exact_sqlite_file — Build lock path targets exact sqlite file	The assertion conditions in the test should pass.	Assertions passed.	PASS

49	test_process_lock.ProcessLockTests.test_windows_probe_error_is_treated_as_not_running — Windows probe error is treated as not running	The assertion conditions in the test should pass.	Assertions passed.	PASS
50	test_raw_writer_dedupe.RawWriterDedupeTests.test_dedupe_survives_reopen_and_keeps_single_row — Dedupe survives reopen and keeps single row	Duplicate input should be rejected or suppressed correctly.	Assertions passed.	PASS
51	test_raw_writer_dedupe.RawWriterDedupeTests.test_existing_legacy_day_file_is_upgraded_with_dew_point_column — Existing legacy day file is upgraded with dew point column	The assertion conditions in the test should pass.	Assertions passed.	PASS
52	test_reassembler.JsonReassemblerTests.test_handles_multiple_objects_and_string_braces — Handles multiple objects and string braces	The assertion conditions in the test should pass.	Assertions passed.	PASS
53	test_resample_grid.ResampleGridTests.test_resample_builds_uniform_grid_and_uses_last_sample_in_bucket — Resample builds uniform grid and uses last sample in bucket	The assertion conditions in the test should pass.	Assertions passed.	PASS
54	test_sqlite_storage.SqliteStorageTests.test_initialize_schema_moves_backfill_sessions_to_negative_range — Initialise schema moves backfill sessions to negative range	Values should stay within the expected range or cadence.	Assertions passed.	PASS
55	test_sqlite_storage.SqliteStorageTests.test_range_query_returns_expected_rows — Range query returns expected rows	The requested output should be returned	Assertions passed.	PASS

		and contain the expected content.		
56	test_sqlite_storage.SqliteStorageTests.test_sequence_restart_reuses_seq_in_new_sqlite_session — Sequence restart reuses seq in new sqlite session	The assertion conditions in the test should pass.	Assertions passed.	PASS
57	test_sqlite_storage.SqliteStorageTests.test_small_sequence_restart_with_higher_uptime_opens_new_session — Small sequence restart with higher uptime opens new session	The assertion conditions in the test should pass.	Assertions passed.	PASS
58	test_sqlite_storage.SqliteStorageTests.test_soft_reload_sequence_drop_with_higher_uptime_opens_new_session — Soft reload sequence drop with higher uptime opens new session	The assertion conditions in the test should pass.	Assertions passed.	PASS
59	test_sqlite_storage.SqliteWriterPipelineTests.test_writer_pipeline_recovers_after_transient_insert_error — Writer pipeline recovers after transient insert error	The assertion conditions in the test should pass.	Assertions passed.	PASS
60	test_validation.ValidationTests.test_validation_flags_missing_values_and_sensor_error — Validation flags missing values and sensor error	The assertion conditions in the test should pass.	Assertions passed.	PASS
61	test_validation.ValidationTests.test_validation_flags_out_of_range_values — Validation flags out of range values	Values should stay within the expected range or cadence.	Assertions passed.	PASS

62	test_watchdog.WatchdogTests.test_duplicate_only_telemetry_does_not_reset_watchdog_progress_timer — Duplicate only telemetry does not reset watchdog progress timer	Duplicate input should be rejected or suppressed correctly.	Assertions passed.	PASS
63	test_watchdog.WatchdogTests.test_watchdog_reconnects_after_resubscribe_does_not_restore_telemetry — Watchdog reconnects after resubscribe does not restore telemetry	The assertion conditions in the test should pass.	Assertions passed.	PASS
64	test_writer_pipeline.WriterPipelineTests.test_queue_pipeline_writes_multiple_records_to_real_csv_outputs — Queue pipeline writes multiple records to real csv outputs	The assertion conditions in the test should pass.	Assertions passed.	PASS
65	test_writer_pipeline.WriterPipelineTests.test_writer_recovers_after_transient_io_error — Writer recovers after transient io error	The assertion conditions in the test should pass.	Assertions passed.	PASS
66	test_evaluator.EvaluatorTests.test_mae_and_rmse_are_computed_correctly — Mae and rmse are computed correctly	The assertion conditions in the test should pass.	Assertions passed.	PASS
67	test_event_detection.EventDetectionTests.test_stable_window_does_not_trigger_event — Stable window does not trigger event	The assertion conditions in the test should pass.	Assertions passed.	PASS
68	test_features.FeatureExtractionTests.test_extract_features_includes_time_and_recent_trends — Extract features includes time and recent trends	The assertion conditions in the test	Assertions passed.	PASS

		should pass.		
69	test_features.FeatureExtractionTests.test_fixture_file_loads_for_small_realistic_window — Fixture file loads for small realistic window	The assertion conditions in the test should pass.	Assertions passed.	PASS
70	test_burst_loss.BurstLossTests.test_burst_mode_increases_drop_and_delay_probability — Burst mode increases drop and delay probability	The assertion conditions in the test should pass.	Assertions passed.	PASS
71	test_event_spike_recovery.EventRecoveryTests.test_event_spike_then_recovers_toward_baseline — Event spike then recovers toward baseline	The assertion conditions in the test should pass.	Assertions passed.	PASS
72	test_generator_ranges.GeneratorRangeTests.test_entrance_profile_changes_stay_gentle_without_forced_events — Entrance profile changes stay gentle without forced events	The assertion conditions in the test should pass.	Assertions passed.	PASS
73	test_generator_ranges.GeneratorRangeTests.test_zone_profiles_stay_within_plausible_bounds — Zone profiles stay within plausible bounds	The assertion conditions in the test should pass.	Assertions passed.	PASS
74	test_schedule_rate.ScheduleRateTests.test_active_hours_use_higher_event_rate — Active hours use higher event rate	The assertion conditions in the test should pass.	Assertions passed.	PASS
75	test_schedule_rate.ScheduleRateTests.test_start_hour_offset_aligns_schedule_to_local	Values should stay	Assertions passed.	PASS

	_clock — Start hour offset aligns schedule to local clock	within the expected range or cadence.		
76	test_timezone_fallback.TimezoneFallbackTests.test_cli_accepts_ianna_timezone_when_zoneinfo_database_is_missing — Cli accepts ianna timezone when zoneinfo database is missing	Fallback or validation logic should behave as intended.	Assertions passed.	PASS
77	test_timezone_fallback.TimezoneFallbackTests.test_cli_still_rejects_obviously_invalid_timezone_names — Cli still rejects obviously invalid timezone names	The assertion conditions in the test should pass.	Assertions passed.	PASS
78	test_timezone_fallback.TimezoneFallbackTests.test_generator_uses_fallback_clock_when_zoneinfo_database_is_missing — Generator uses fallback clock when zoneinfo database is missing	Fallback or validation logic should behave as intended.	Assertions passed.	PASS
79	test_weather_trend.WeatherTrendTests.test_daytime_target_is_warmer_and_drier_than_pre_dawn — Daytime target is warmer and drier than pre dawn	The assertion conditions in the test should pass.	Assertions passed.	PASS
80	test_weather_trend.WeatherTrendTests.test_summer_target_is_warmer_than_winter_target — Summer target is warmer than winter target	The assertion conditions in the test should pass.	Assertions passed.	PASS

Table 4. *Rest of Test cases*